

第7章 死 锁

在多道程序环境下，多个进程可能竞争一定数量的资源。某个进程申请资源，如果这时资源不可用，那么该进程进入等待状态。如果所申请的资源被其他等待进程占有，那么该等待进程有可能再也无法改变其状态。这种情况称为死锁（deadlock）。第6章已经结合信号量讨论了这类情况。

最好的死锁例子可能是 Kansas 立法机构于 20 世纪初通过的一个法规，其中说到“当两列列车在十字路口逼近时，它们要完全停下来，且在一列列车开走之前，另一列列车不能启动。”

本章介绍一些方法，以便操作系统用于预防或处理死锁。虽然现在绝大多数操作系统并不提供死锁预防功能，但是这些功能可能很快会加入。随着以下趋势的不断发展：更多数量的进程、多线程程序、更多的系统资源、持久文件和数据库服务越来越被重视（而不是批处理），死锁问题会变得更普遍。

本章目标

- 描述死锁现象，即使得一组并发进程不能完成其任务的现象。
- 提供一些方法以预防或避免计算机系统内的死锁。

7.1 系 统 模 型

某系统拥有一定数量的资源，分布在若干竞争进程之间。这些资源可分成多种类型，每种类型有一定数量的实例。资源类型的例子有内存空间、CPU 周期、文件、I/O 设备（打印机和 DVD 驱动器）等。如果系统有两个 CPU，那么资源类型 *CPU* 就有两个实例。类似地，资源类型 *打印机* 可能有 5 个实例。

如果一个进程申请某个资源类型的一个实例，那么分配这种类型的任何实例都可满足申请。否则，这些实例就不相同，且资源类型的分类也没有正确定义。例如，一个系统有两台打印机。如果没有人关心哪台打印机打印哪些输出，那么这两台打印机可定义为属于同一资源类型。然而，如果一台打印机在九楼，而另一台在一楼，那么九楼的用户就不会认为这两台打印机是相同的，这样每个打印机就可能需要定义为属于不同类型。

进程在使用资源前必须申请资源，在使用资源之后必须释放资源。一个进程可能会申

请许多资源以便完成其指定的任务。显然，所申请的资源数量不能超过系统所有资源的总量。换言之，如果系统只有两台打印机，那么进程就不能申请三台打印机。

在正常操作模式下，进程只能按如下顺序使用资源：

① **申请**：如果申请不能立即被允许（例如，所申请资源正在为其他进程所使用），那么申请进程必须等待，直到它获得该资源为止。

② **使用**：进程对资源进行操作（例如，如果资源是打印机，那么进程就可以在打印机上打印了）。

③ **释放**：进程释放资源。

如第 2 章所述，资源的申请与释放为系统调用，例如系统调用：`request()/release()`（设备）、`open()/close()`（文件）、`allocate()/free()`（内存）。其他资源的申请与释放可以通过信号量的 `wait` 与 `signal` 操作或通过互斥锁的获取与释放来完成。因此，对于进程或线程的每次使用，操作系统会检查以确保使用进程已经申请并获得了资源。系统表记录了每个资源是否空闲或已被分配，分配给了哪个进程。如果进程所申请的资源正在为其他进程所使用，那么该进程会增加到该资源的等待队列。

当一组进程中的每个进程都在等待一个事件，而这一事件只能由这一组进程的另一进程引起，那么这组进程就处于死锁状态。这里所关心的主要事件是资源获取和释放。资源可能是物理资源（例如，打印机、磁带驱动器、内存空间和 CPU 周期）或逻辑资源（例如，文件、信号量和管程）。然而，其他类型事件也会导致死锁（例如，第 4 章所讨论的 IPC 功能）。

为说明死锁状态，以一个具有三个 CD 刻录机的系统为例。假定有三个进程，每个进程都占用了—个 CD 刻录机。如果每个进程现在需要另一个刻录机，那么这三个进程会处于死锁状态。每个进程都在等待事件“CD 刻录机释放”，这只能由一个等待进程来完成。这个例子说明了涉及同一种资源类型的死锁。

死锁也可能涉及不同资源类型。例如，某一个系统有一台打印机和一台 DVD 驱动器。假如进程 P_i 占有 DVD 驱动器而 P_j 占有打印机，如果 P_i 申请打印机而 P_j 申请 DVD 驱动器，那么就会出现死锁。

开发多线程应用程序的程序员必须特别关注这个问题，因为多个线程可能因为竞争共享资源而容易产生死锁。

7.2 死 锁 特 征

当出现死锁时，进程永远不能完成，并且系统资源被阻碍使用，阻止了其他作业开始执行。在讨论处理死锁问题的各种方法之前，先深入讨论一下死锁的特征。

7.2.1 必要条件

如果在一个系统中下面 4 个条件同时满足，那么会引起死锁。

① **互斥**：至少有一个资源必须处于非共享模式，即一次只有一个进程使用。如果另一进程申请该资源，那么申请进程必须等到该资源被释放为止。

② **占有并等待**：一个进程必须占有至少一个资源，并等待另一资源，而该资源为其他进程所占有。

③ **非抢占**：资源不能被抢占，即资源只能在进程完成任务后自动释放。

④ **循环等待**：有一组等待进程 $\{P_0, P_1, \dots, P_n\}$ ， P_0 等待的资源为 P_1 所占有， P_1 等待的资源为 P_2 所占有，……， P_{n-1} 等待的资源为 P_n 所占有， P_n 等待的资源为 P_0 所占有。

在此，强调所有 4 个条件必须同时满足才会出现死锁。循环等待条件意味着占有并等待条件，这样 4 个条件并不完全独立。然而，在 7.4 节中将会看到分开考虑这些条件是有意义的。

互斥锁中的死锁

下面来看在一个使用互斥锁的 **Pthread** 多线程程序中的死锁。`pthread_mutex_init()` 函数初始化一个未锁的互斥体。互斥锁分别通过 `pthread_mutex_lock()` 以及 `pthread_mutex_unlock()` 来获取和释放。如果一个线程试图获取一个已加锁的互斥体，`pthread_mutex_lock()` 调用就会阻塞该线程，直到互斥锁所有者调用 `pthread_mutex_unlock()` 为止。

下面的代码例子创建了两个互斥锁：

```
/* Create and initialize the mutex locks */
pthread_mutex_t first_mutex;
pthread_mutex_t second_mutex;

pthread_mutex_init(&first_mutex, NULL);
pthread_mutex_init(&second_mutex, NULL);
```

下一步创建两个线程 `thread_one` 和 `thread_two`，这两个线程都能访问这两个互斥锁。`thread_one` 和 `thread_two` 分别运行在 `do_work_one()` 和 `do_work_two()` 函数中，如图 7.1 所示。

在这个例子中，`thread_one` 试图以顺序(1)`first_mutex`、(2)`second_mutex` 来获取互斥锁，`thread_two` 试图以顺序(1)`second_mutex`、(2)`first_mutex` 来获取互斥锁。如果 `thread_one` 获得了 `first_mutex`，同时 `thread_two` 获得了 `second_mutex`，那么就可能发生死锁。

注意，虽然死锁的发生是可能的，但是如果 `thread_one` 能够在 `thread_two` 试图获得锁之前，获得并释放互斥锁 `first_mutex` 和 `second_mutex`，那么死锁仍然不会发生。这个例子说明了处理死锁的一个问题：很难识别并测试只在某种情况下发生的死锁。

```

/* thread_one runs in this function */
void *do_work_one(void *param)
{
    pthread_mutex_lock(&first_mutex);
    pthread_mutex_lock(&second_mutex);
    /**
     * Do some work
     */
    pthread_mutex_unlock(&second_mutex);
    pthread_mutex_unlock(&first_mutex);

    pthread_exit(0);
}

/* thread_two runs in this function */
void *do_work_two(void *param)
{
    pthread_mutex_lock(&second_mutex);
    pthread_mutex_lock(&first_mutex);
    /**
     * Do some work
     */
    pthread_mutex_unlock(&first_mutex);
    pthread_mutex_unlock(&second_mutex);

    pthread_exit(0);
}

```

图 7.1 死锁例子

7.2.2 资源分配图

死锁问题可用称为系统资源分配图的有向图进行更为精确地描述。这种图由一个节点集合 V 和一个边集合 E 组成。节点集合 V 可分成两种类型的节点： $P=\{P_1, P_2, \dots, P_n\}$

(系统活动进程的集合) 和 $R=\{R_1, R_2, \dots, R_m\}$ (系统所有资源类型的集合)。

由进程 P_i 到资源类型 R_j 的有向边记为 $P_i \rightarrow R_j$, 它表示进程 P_i 已经申请了资源类型 R_j 的一个实例, 并正在等待该资源。由资源类型 R_j 到进程 P_i 的有向边记为 $R_j \rightarrow P_i$, 它表示资源类型 R_j 的一个实例已经分配给进程 P_i 。有向边 $P_i \rightarrow R_j$ 称为**申请边**, 有向边 $R_j \rightarrow P_i$ 称为**分配边**。

在图上, 用圆形表示进程 P_i , 用矩形表示资源类型 R_j 。由于资源类型 R_j 可能有多个实例, 所以在矩形中用圆点表示实例。注意申请边只指向矩形 R_j , 而分配边必须指定矩形内的某个圆点。

当进程 P_i 申请资源类型 R_j 的一个实例时, 就在资源分配图中加入一条申请边。当该申请可以得到满足时, 那么申请边就立即转换成分配边。当进程不再需要访问资源时, 它就释放资源, 因此就删除分配边。

图 7.2 的资源分配图表示了如下情况:

- 集合 P 、 R 和 E :
 - $P=\{P_1, P_2, P_3\}$ 。
 - $R=\{R_1, R_2, R_3, R_4\}$ 。
 - $E=\{ P_1 \rightarrow R_1, P_2 \rightarrow R_3, R_1 \rightarrow P_2, R_2 \rightarrow P_2, R_2 \rightarrow P_1, R_3 \rightarrow P_3 \}$ 。
- 资源实例:
 - 资源类型 R_1 有 1 个实例。
 - 资源类型 R_2 有 2 个实例。
 - 资源类型 R_3 有 1 个实例。
 - 资源类型 R_4 有 3 个实例。
- 进程状态:
 - 进程 P_1 占有资源类型 R_2 的 1 个实例, 等待资源类型 R_1 的 1 个实例。
 - 进程 P_2 占有资源类型 R_1 的 1 个实例和资源类型 R_2 的 1 个实例, 等待资源类型 R_3 的 1 个实例。
 - 进程 P_3 占有资源类型 R_3 的 1 个实例。

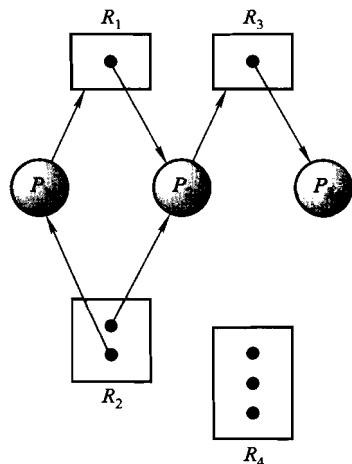


图 7.2 资源分配图

根据资源分配图的定义, 可以证明: 如果分配图没有环, 那么系统就没有进程死锁。如果分配图有环, 那么可能存在死锁。

如果每个资源类型刚好有一个实例, 那么有环就意味着已经出现死锁。如果环涉及一组资源类型, 而每个类型只有一个实例, 那么就出现死锁。环所涉及的进程就死锁。在这种情况下, 图中的环就是死锁存在的充分必要条件。

如果每个资源类型有多个实例，那么有环并不意味着已经出现了死锁。在这种情况下，图中的环就是死锁存在的必要条件而不是充分条件。

为了说明这个概念，下面回到图 7.2 所示的资源分配图。假设进程 P_3 申请了资源类型 R_2 的一个资源。由于现在没有资源实例可用，所以就增加了有向边 $P_3 \rightarrow R_2$ （见图 7.3）。这时，在系统中有两个最小环：

$$P_1 \rightarrow R_1 \rightarrow P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$$

$$P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_2$$

进程 P_1 、 P_2 和 P_3 是死锁。进程 P_2 等待资源类型 R_3 ，而它又被进程 P_3 占有。另一方面，进程 P_3 等待进程 P_1 或进程 P_2 以释放资源类型 R_2 。另外，进程 P_1 等待进程 P_2 释放资源 R_1 。

现在考虑图 7.4 所示的资源分配图。在这个例子中，也有一个环

$$P_1 \rightarrow R_1 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$$

然而，却没有死锁。注意进程 P_4 可能释放资源类型 R_2 的实例。这个资源可分配给进程 P_3 ，以打破环。

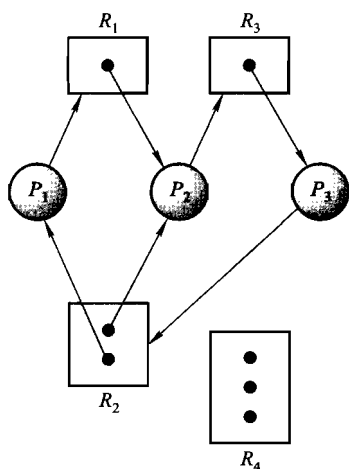


图 7.3 存在死锁的资源分配图

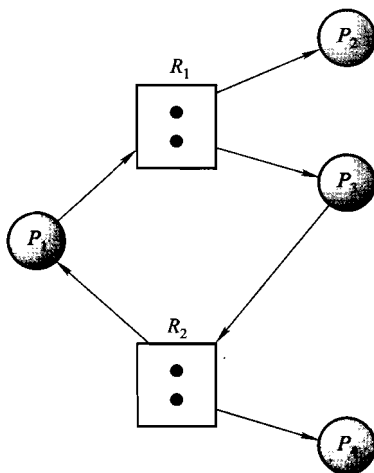


图 7.4 存在环但是没有死锁的资源分配图

总而言之，如果资源分配图没有环，那么系统就不处于死锁状态。另一方面，如果有环，那么系统可能会处于死锁状态。在处理死锁问题时，这点是很重要的。

7.3 死锁处理方法

从原理上来说，有三种方法可处理死锁问题：

- 可使用协议以预防或避免死锁，确保系统不会进入死锁状态。
- 可允许系统进入死锁状态，然后检测它，并加以恢复。
- 可忽视这个问题，认为死锁不可能在系统内发生。

这里第三种方法为绝大多数操作系统所采用，包括 UNIX 和 Windows。因此应用程序开发人员需要自己来处理死锁。

接下来，将简单描述每种死锁处理方法。然后，在 7.4 节到 7.7 节中，将详细讨论每种算法。不过，在开始之前，有必要说明有些科研人员认为这些基本方法不能单独用来处理操作系统的所有资源分配问题。然而，可以将这些基本方法组合起来，以便为每种系统资源选择一种最佳方法。

为了确保死锁不会发生，系统可以采用死锁预防或死锁避免方案。**死锁预防**（deadlock prevention）是一组方法，以确保至少一个必要条件不成立（参见 7.2.1 小节）。这些方法通过限制如何申请资源的方法来预防死锁。在 7.4 节中将讨论这些方法。

死锁避免（deadlock avoidance）要求操作系统事先得到有关进程申请资源和使用资源的额外信息。有了这些额外信息，系统可确定：对于一个申请，进程是否应等待。为了确定当前申请是允许还是延迟，系统必须考虑现有可用资源、已分配给每个进程的资源、每个进程将来申请和释放的资源。在 7.5 节中将讨论这些方案。

如果系统不使用死锁预防或死锁避免算法，那么可能发生死锁情况。在这种情况下，系统可提供一个算法来检查系统状态以确定死锁是否发生，并提供另一个算法来从死锁中恢复（如果死锁确实已发生）。在 7.6 和 7.7 节中将讨论这些问题。

如果系统不能确保不会发生死锁，且也不提供机制进行死锁检测和恢复，那么可能出现这种情况：系统处于死锁而又没有办法检测到发生了什么。在这种情况下，未检查死锁会导致系统性能下降，因为资源被不能运行的进程所占有，而越来越多进程会因申请资源而进入死锁。最后，整个系统会停止工作，并且需要人工重新启动。

虽然这看起来似乎不是一个解决死锁问题的可行方法，但是它却为某些操作系统所使用。对许多系统，死锁很少发生（如一年一次），因此，与使用频繁的并且开销昂贵的死锁预防、死锁避免和死锁检测与恢复相比，这种方法更为便宜。而且，在有的情况下，系统处于冻结状态而不是死锁状态。比如，一个实时进程运行于最高优先级（或其他进程运行于非抢占调用程序之下）且不将控制返回给操作系统。因此，系统必须人工地从非死锁状态中恢复，这也是死锁恢复采用的技术。

7.4 死锁预防

如 7.2.1 小节所述，出现死锁有 4 个必要条件。只要确保至少一个必要条件不成立，就能预防死锁发生。下面通过详细讨论这 4 个必要条件来研究死锁预防方法。

7.4.1 互斥

对于非共享资源，必须要有互斥条件。例如，一台打印机不能同时为多个进程所共享。另一方面，共享资源不要求互斥访问，因此不会涉及死锁。共享资源的一个很好的例子是只读文件。如果多个进程试图同时打开只读文件，那么它们能同时获得对只读文件的访问。进程决不需要等待共享资源。然而，通常不能通过否定互斥条件来预防死锁：有的资源本身就是非共享的。

7.4.2 占有并等待

为了确保占有并等待条件不会在系统内出现，必须保证：当一个进程申请一个资源时，它不能占有其他资源。一种可以使用的协议是每个进程在执行前申请并获得所有资源。可以实现通过要求申请资源的系统调用在所有其他系统调用之前进行。

另外一种协议允许进程在没有资源时才可申请资源。一个进程可申请一些资源并使用它们。然而，在它申请更多其他资源之前，它必须释放其现已分配的所有资源。

为了说明这两种协议之间的差别，考虑一个进程，它将数据从 DVD 驱动器复制到磁盘文件，并对磁盘文件进行排序，再将结果打印到打印机上。如果所有资源必须在进程开始之前申请，那么进程必须一开始就申请 DVD 驱动器、磁盘文件和打印机。在其整个执行过程中，它会一直占有打印机，尽管它只在结束时才需要打印机。

第二种方法允许进程在开始时只申请 DVD 驱动器和磁盘文件。它将数据从 DVD 复制到磁盘，再释放 DVD 驱动器和磁盘文件。然后，进程必须再申请磁盘文件和打印机。当数据从磁盘文件复制到打印机之后，它就释放这两个资源并终止。

这两种协议有两个主要缺点。第一，资源利用率 (resource utilization) 可能比较低，因为许多资源可能已分配，但是很长时间没有被使用。例如，在所给的例子中，只有确认数据始终在磁盘文件上的情况下，才可以释放 DVD 驱动器和磁盘文件，并再次申请磁盘文件和打印机资源。否则，不管采用哪种协议，必须在开始之前申请所有资源。第二，可能发生饥饿。一个进程如需要多个常用资源，可能会永久等待，因为其所需要的资源中至少有一个已分配给其他进程。

7.4.3 非抢占

第三个必要条件是对已分配的资源不能抢占。为了确保这一条件不成立，可以使用如下协议：如果一个进程占有资源并申请另一个不能立即分配的资源，那么其现已分配的资源都可被抢占。换句话说，这些资源都被隐式地释放了。抢占资源分配到进程所等待的资源的链表上。只有当进程获得其原有资源和所申请的新资源时，进程才可以重新执行。

换句话说，如果一个进程申请一些资源，那么首先检查它们是否可用。如果可用，那

么就分配它们。如果不可用，那么检查这些资源是否已分配给其他等待额外资源的进程。如果是，那么就从等待进程中抢占这些资源，并分配给申请进程。如果资源不可用且也不被其他等待进程占有，那么申请进程必须等待。当一个进程处于等待时，如果其他进程申请其拥有资源，那么该进程的部分资源可以被抢占。一个进程要重新执行，它必须分配到其所申请的资源，并恢复其在等待时被抢占的资源。

这个协议通常应用于状态可以保存和恢复的资源，如 CPU 寄存器和内存。它一般不适用于其他资源，如打印机和磁带驱动器。

7.4.4 循环等待

死锁的第 4 个也是最后一个条件是循环等待。一个确保此条件不成立的方法是对所有资源类型进行完全排序，且要求每个进程按递增顺序来申请资源。

设 $R=\{R_1, R_2, \dots, R_m\}$ 为资源类型的集合。为每个资源类型分配一个唯一整数来允许比较两个资源以确定其先后顺序。可以定义一个函数 $F: R \rightarrow N$ ，其中 N 是自然数的集合。例如，如果资源类型 R 的集合包括磁带驱动器、磁盘驱动器和打印机，那么函数 F 可以定义如下：

```
F(tape drive)=1
F(disk drive)=5
F(printer) =12
```

可以采用如下协议以预防死锁：每个进程只按递增顺序申请资源，即一个进程开始可申请任何数量的资源类型 R_i 的实例。之后，当且仅当 $F(R_j) > F(R_i)$ 时，该进程可以申请资源类型 R_j 的实例。如果需要同一资源类型的多个实例，那么对它们必须一起申请。例如，对于以上给定函数，一个进程需要同时使用磁带驱动器和打印机，那么就必须先申请磁带驱动器，再申请打印机。换句话说，要求当一个进程申请资源类型 R_j 时，它必须先释放所有资源 R_i ($F(R_i) \geq F(R_j)$)。

如果使用这两个协议，那么循环等待就不可能成立。可以通过反证法来证明这一点。假定有一个循环等待存在。设涉及循环等待的进程集合为 $\{P_0, P_1, \dots, P_n\}$ ，其中 P_i 等待一个资源 R_i ，而 R_i 又为进程 P_{i+1} 所占有（对于索引采用模数代数，因此 P_n 等待由 P_0 所占有的资源 R_n ）。因此，由于进程 P_{i+1} 占有资源 R_i 而同时申请资源 R_{i+1} ，所以对所有 i ，必须有 $F(R_i) < F(R_{i+1})$ 。而这意味着 $F(R_0) < F(R_1) < \dots < F(R_n) < F(R_0)$ 。根据传递规则， $F(R_0) < F(R_0)$ ，这显然是不可能的。因此，不可能有循环等待。

通过对系统内所有同步对象进行完全排序，可以在应用程序中实现这种方案。所有这些同步对象的请求必须按递增顺序进行。例如，如果将图 7.1 所示的 Pthread 程序的锁的顺序定义为：

```
F(first_mutex) = 1
F(second_mutex) = 5
```

那么 `thread_two` 不能按错误顺序来申请锁。

请记住，设计一个完全排序或层次并不能防止死锁，而是要靠应用程序员来按顺序编写程序。另外，函数 F 应该根据系统内资源使用的正常顺序来定义。例如，由于磁带通常在打印机之前使用，所以定义 $F(\text{tape drive}) < F(\text{printer})$ 较为合理。

虽然应用程序员有责任确保按适当顺序获取资源，但是有些软件可以用来验证锁是否按顺序获取。如果不按顺序申请且可能出现死锁，那么这些软件会给出适当的警告。`Witness` 就是一种锁顺序验证器，它运行于 BSD UNIX 上类似于 FreeBSD。`Witness` 采用互斥锁来保护临界区域，如第 6 章所述；它通过动态维护系统内的锁顺序来工作。下面通过图 7.1 所示的例子来说明。假设 `thread_one` 首先获取锁，并按如下顺序：(1)`first_mutex`，(2)`second_mutex`。`Witness` 记录这种关系：获取 `first_mutex` 必须在获取 `second_mutex` 之前进行。如果 `thread_two` 后来不按这个顺序来获取锁，那么 `witness` 就在系统终端上生成一个警告信息。

7.5 死 锁 避 免

在 7.4 节讨论的死锁预防算法中，通过限制资源申请的方法来预防死锁。这种限制确保 4 个必要条件之一不会发生，因此死锁不成立。然而，通过这种方法预防死锁的副作用是低设备使用率和系统吞吐率。

避免死锁的另一种方法是，获得以后如何申请资源的附加信息。例如，对于有一台磁带驱动器和一台打印机的系统，可能知道进程 P 会先申请磁带驱动器，再申请打印机，之后释放这些资源。另一方面，进程 Q 会先申请打印机，再申请磁带驱动器。有了关于每个进程的申请与释放的完全顺序，可决定进程是否因申请而等待。每次申请要求系统考虑现有可用资源、现已分配给每个进程的资源 and 每个进程将来申请与释放的资源，以决定当前申请是否满足或必须等待，从而避免死锁发生的可能性。

根据这种方法，有不同的算法，它们在所要求的信息量和信息的类型上有所不同。最为简单和最为有用的模型要求每个进程说明可能需要的每种资源类型实例的**最大需求**。根据每个进程可能申请的每种资源类型实例的最大需求的事先信息，可以构造一个算法以确保系统决不会进入死锁状态。这种算法定义了**死锁避免**（`deadlock-avoidance`）方法。**死锁避免**算法动态地检测资源分配状态以确保循环等待条件不可能成立。资源分配状态是由可用资源和已分配资源，及进程最大需求所决定的。下面来研究两个死锁避免算法。

7.5.1 安全状态

如果系统能按某个顺序为每个进程分配资源（不超过其最大值）并能避免死锁，那么系统状态就是安全的。更为准确地说，如果存在一个**安全序列**，那么系统处于安全状态。

进程顺序 $\langle P_1, P_2, \dots, P_n \rangle$ ，如果对于每个 P_i ， P_i 仍然可以申请的资源数小于当前可用资源加上所有进程 P_j （其中 $j < i$ ）所占有的资源，那么这一顺序称为安全序列。在这种情况下，进程 P_i 所需要的资源即使不能立即可用，那么 P_i 可等待直到所有 P_j 释放其资源。当它们完成时， P_i 可得到其所需要的所有资源，完成其给定任务，返回其所分配的资源并终止。当 P_i 终止时， P_{i+1} 可得到其所需要的资源，如此进行。如果没有这样的顺序存在，那么系统状态就处于不安全状态。

安全状态不是死锁状态。相反，死锁状态是不安全状态。然而，不是所有不安全状态都能导致死锁状态（见图 7.5）。不安全状态可能死锁。只要状态为安全，操作系统就能避免不安全（和死锁）状态。在不安全状态下，操作系统不能阻止进程以会导致死锁的方式申请资源。进程行为控制了不安全状态。

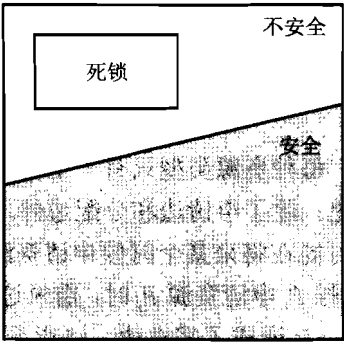


图 7.5 安全、不安全和死锁状态空间

例如，考虑一个系统，有 12 台磁带驱动器和三个进程 P_0 、 P_1 、 P_2 。进程 P_0 最多要求 10 台磁带驱动器， P_1 最多要求 4 台磁带驱动器， P_2 最多要求 9 台磁带驱动器。假定，在时间 t_0 时，进程 P_0 占有 5 台磁带驱动器，进程 P_1 占有 2 台磁带驱动器，进程 P_2 占有 2 台磁带驱动器（因此，还有 3 台空闲磁带驱动器）。

	<u>最大需求</u>	<u>当前需求</u>
P_0	10	5
P_1	4	2
P_2	9	2

在时刻 t_0 ，系统处于安全状态。顺序 $\langle P_1, P_0, P_2 \rangle$ 满足安全条件，这是因为进程 P_1 可立即得到其所有磁带驱动器并接着归还它们（系统会有 5 台磁带驱动器），接着进程 P_0 可得到其所有磁带驱动器并归还它们（这时系统会有 10 台磁带驱动器），最后进程 P_2 得到其所有磁带驱动器并归还它们（系统会有 12 台磁带驱动器）。

系统可以从安全状态转换为不安全状态。假定在时刻 t_1 ，进程 P_2 申请并又得到了 1 台

磁带驱动器, 系统就不再安全了。这时, 只有进程 P_1 可得到其所有磁带驱动器。当其返回这些资源时, 系统只有 4 台磁带驱动器可用。由于进程 P_0 已分配了 5 台磁带驱动器而其最大需求为 10 台磁带驱动器, 所以它还需要 5 台磁带驱动器。因为现在不够, 所以进程 P_0 必须等待。类似地, 进程 P_2 还需要 6 台磁带驱动器, 也必须等待, 导致了死锁。这时的错误在于允许进程 P_2 再获得 1 台磁带驱动器。如果让 P_2 等待直到其他进程之一完成并释放其资源, 那么就能避免死锁。

有了安全状态的概念, 可定义避免算法以确保系统不会死锁。其思想是简单地确保系统始终处于安全状态。开始, 系统处于安全状态。当进程申请一个可用的资源时, 系统必须确定这一资源申请是可以立即分配还是要等待。只有分配后使系统仍处于安全状态, 才允许申请。

采用这种方案, 如果进程申请一个现已可用的资源, 那么它可能必须等待。因此, 与没有采用死锁避免算法相比, 这种情况下的资源使用率可能更低。

7.5.2 资源分配图算法

如果有一个资源分配系统, 每种资源类型只有一个实例, 那么 7.2.2 小节所定义的资源分配图可修改以用于死锁避免。除了申请边和分配边外, 引入一新类型的边, 称为需求边。需求边 $P_i \rightarrow R_j$ 表示进程 P_i 可能在将来某个时候申请资源 R_j 。这种边类似于同一方向的申请边, 但是用虚线表示。当进程 P_i 申请资源 R_j 时, 需求边 $P_i \rightarrow R_j$ 变成了申请边。类似地, 当进程 P_i 释放 R_j 时, 分配边 $R_j \rightarrow P_i$ 变成了需求边。注意系统必须事先说明所要求的资源, 即当进程 P_i 开始执行时, 所有需求边必须先处于资源分配图。可放宽这个条件, 以允许只有在进程 P_i 的所有相关的边都为需求边时才将需求边 $P_i \rightarrow R_j$ 增加到图中。

假设进程 P_i 申请资源 R_j 。只有在将申请边 $P_i \rightarrow R_j$ 变成分配边 $R_j \rightarrow P_i$ 而不会导致资源分配图形成环时, 才允许申请。注意, 通过采用环检测算法, 检测安全性。检测图中是否有环的算法需要 n^2 级的操作, 其中 n 是系统的进程数量。

如果没有环存在, 那么资源分配会使得系统处于安全状态。如果有环存在, 那么分配会导致系统处于不安全状态。因此, 进程 P_i 必须等待其资源申请被满足。

为了说明这个算法, 考虑图 7.6 所示的资源分配图。假设进程 P_2 申请资源 R_2 。虽然 R_2 现在可用, 但是不能将它分配给 P_2 , 因为这会创建一个环 (如图 7.7 所示)。环表示系统处于不安全状态。如果 P_1 申请 R_2 且 P_2 申请 R_1 , 那么会发生死锁。

7.5.3 银行家算法

对于每种资源类型有多个实例的资源分配系统, 资源分配图算法就不适用了。下面所要描述的死锁避免算法适用于这种系统, 但是其效率要比资源分配图方案低。这一算法通常称为银行家算法。该算法如此命名是因为它可用于银行系统, 当它不能满足所有客户的

需要时, 银行决不会分配其现金。

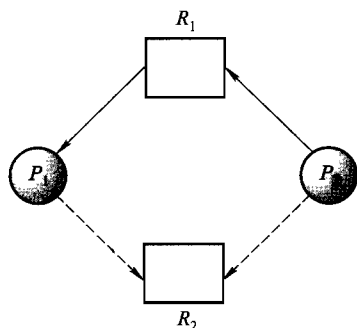


图 7.6 死锁避免的资源分配图

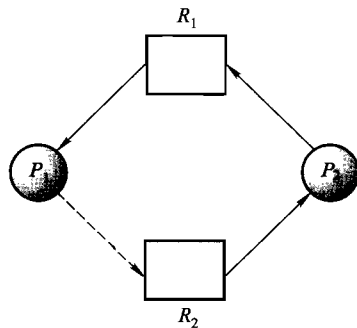


图 7.7 资源分配图的不安全状态

当新进程进入系统时, 它必须说明其可能需要的每种类型资源实例的最大数量, 这一数量不能超过系统资源的总和。当用户申请一组资源时, 系统必须确定这些资源的分配是否仍会使系统处于安全状态。如果是, 就可分配资源; 否则, 进程必须等待直到某个其他进程释放足够资源为止。

为了实现银行家算法, 必须有几个数据结构。这些数据结构对资源分配系统的状态进行了记录。设 n 为系统进程的个数, m 为资源类型的种类。需要如下数据结构:

- **Available:** 长度为 m 的向量, 表示每种资源的现有实例的数量。如果 $Available[j]=k$, 那么资源类型 R_j 现有 k 个实例。

- **Max:** $n \times m$ 矩阵, 定义每个进程的最大需求。如果 $Max[i][j]=k$, 那么进程 P_i 最多可申请 k 个资源类型 R_j 的实例。

- **Allocation:** $n \times m$ 矩阵, 定义每个进程现在所分配的各种资源类型的实例数量。如果 $Allocation[i][j]=k$, 那么进程 P_i 现在已分配了 k 个资源类型 R_j 的实例。

- **Need:** $n \times m$ 矩阵, 表示每个进程还需要的剩余的资源。如果 $Need[i][j]=k$, 那么进程 P_i 还可能申请 k 个资源类型 R_j 的实例。注意, $Need[i][j] = Max[i][j] - Allocation[i][j]$ 。

这些数据结构的大小和值会随着时间而改变。

为了简化银行家算法的描述, 可以采用一些符号。设 X 和 Y 为长度为 n 的向量, 则 $X \leq Y$ 当且仅当对所有 $i=1, 2, \dots, n$, $X[i] \leq Y[i]$ 。例如, 如果 $X=(1,7,3,2)$ 而 $Y=(0,3,2,1)$, 那么 $Y \leq X$ 。如果 $Y \leq X$ 且 $Y \neq X$, 那么 $Y < X$ 。

可以将矩阵 $Allocation$ 和 $Need$ 的每行作为向量, 并分别用 $Allocation_i$ 和 $Need_i$ 来表示。向量 $Allocation_i$ 表示分配给进程 P_i 的资源; 向量 $Need_i$ 表示进程为完成其任务可能仍然需要申请的额外资源。

1. 安全性算法

确定计算机系统是否处于安全状态的算法分为如下几步：

① 设 $Work$ 和 $Finish$ 分别为长度为 m 和 n 的向量。按如下方式进行初始化， $Work=Available$ 且 对于 $i=0,1,\dots,n-1, Finish[i]=false$ 。

② 查找这样的 i 使其满足

a. $Finish[i] = false$

b. $Need_i \leq Work$

如果没有这样的 i 存在，那么就转到第④步。

③ $Work = Work + Allocation_i$

$Finish[i] = true$

返回到第②步。

④ 如果对所有 $i, Finish[i]=true$ ，那么系统处于安全状态。

这个算法可能需要 $m \times n^2$ 数量级的操作以确定系统状态是否安全。

2. 资源请求算法

现在，描述如何判断是否可安全允许请求的算法。

设 $Request_i$ 为进程 P_i 的请求向量。如果 $Request_i[j] = k$ ，那么进程 P_i 需要资源类型 R_j 的实例数量为 k 。当进程 P_i 作出资源请求时，采取如下动作。

① 如果 $Request_i \leq Need_i$ ，那么转到第②步。否则，产生出错条件，这是因为进程 P_i 已超过了其最大请求。

② 如果 $Request_i \leq Available$ ，那么转到第③步。否则， P_i 必须等待，这是因为没有可用资源。

③ 假定系统可以分配给进程 P_i 所请求的资源，并按如下方式修改状态：

$Available = Available - Request_i$;

$Allocation_i = Allocation_i + Request_i$;

$Need_i = Need_i - Request_i$;

如果所产生的资源分配状态是安全的，那么交易完成且进程 P_i 可分配到其所需要资源。然而，如果新状态不安全，那么进程 P_i 必须等待 $Request_i$ 并恢复到原来资源分配状态。

3. 举例

最后，为了说明银行家算法的使用，考虑这样一个系统，有 5 个进程 $P_0 \sim P_4$ ，3 种资源类型 A, B, C 。资源类型 A 有 10 个实例，资源类型 B 有 5 个实例，资源类型 C 有 7 个实例。假定在时刻 T_0 ，系统状态如下：

	<u>Allocation</u>			<u>Max</u>			<u>Available</u>		
	<i>A</i>	<i>B</i>	<i>C</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>A</i>	<i>B</i>	<i>C</i>
P_0	0	1	0	7	5	3	3	3	2
P_1	2	0	0	3	2	2			
P_2	3	0	2	9	0	2			
P_3	2	1	1	2	2	2			
P_4	0	0	2	4	3	3			

矩阵 *Need* 的内容定义成 *Max-Allocation*:

	<u>Need</u>		
	<i>A</i>	<i>B</i>	<i>C</i>
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

可以认为系统现在处于安全状态。事实上, 顺序 $\langle P_1, P_3, P_4, P_2, P_0 \rangle$ 满足安全条件。现在假定进程 P_1 再请求 1 个资源类型 *A* 和 2 个资源类型 *C*, 这样 $Request_1 = (1, 0, 2)$ 。为了确定这个请求是否可以立即允许, 首先检测 $Request_1 \leq Available$ (即, $(1, 0, 2) \leq (3, 3, 2)$), 其值为真。接着假定这个请求被满足, 会产生如下新状态:

	<u>Allocation</u>			<u>Need</u>			<u>Available</u>		
	<i>A</i>	<i>B</i>	<i>C</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>A</i>	<i>B</i>	<i>C</i>
P_0	0	1	0	7	4	3	2	3	0
P_1	3	0	2	0	2	0			
P_2	3	0	2	6	0	0			
P_3	2	1	1	0	1	1			
P_4	0	0	2	4	3	1			

必须确定这个状态是否安全。为此, 执行安全算法, 并找到顺序 $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ 满足安全要求。因此, 可以立即允许进程 P_1 的这个请求。

然而, 可以发现当系统处于这一状态时, 是不能允许 P_4 的请求 $(3, 3, 0)$ 的, 因为没有这么多资源可用。也不能允许 P_0 的请求 $(0, 2, 0)$: 虽然有资源可用, 但是这会导致系统处于不安全状态。

如何实现银行家算法, 这就留给读者作为一个编程练习。

7.6 死 锁 检 测

如果一个系统既不采用死锁预防算法也不采用死锁避免算法，那么可能会出现死锁。在这种环境下，系统应提供：

- 一个用来检查系统状态从而确定是否出现了死锁的算法。
- 一个用来从死锁状态中恢复的算法。

在以下讨论中，将针对每种资源类型只有单个实例和每种资源类型可有多个实例这两种情况，并分别研究这两个算法。不过，现在需要注意到检测并恢复方案会有额外开销，这些不但包括维护所需信息和执行检测算法的运行开销，而且也包括死锁恢复所引起的损失。

7.6.1 每种资源类型只有单个实例

如果所有资源类型只有单个实例，那么可以定义这样一个死锁检测算法，该算法使用了资源分配图的一个变种，称为等待（wait-for）图。从资源分配图中，删除所有资源类型节点，合并适当边，就可以得到等待图。

更确切地说，等待图中的由 P_i 到 P_j 的边意味着进程 P_i 等待进程 P_j 释放一个 P_i 所需的资源。等待图有一条 $P_i \rightarrow P_j$ 的边，当且仅当相应的资源分配图中包含两条边 $P_i \rightarrow R_q$ 和 $R_q \rightarrow P_j$ ，其中 R_q 为资源。例如，图 7.8 显示了资源分配图和对应的等待图。

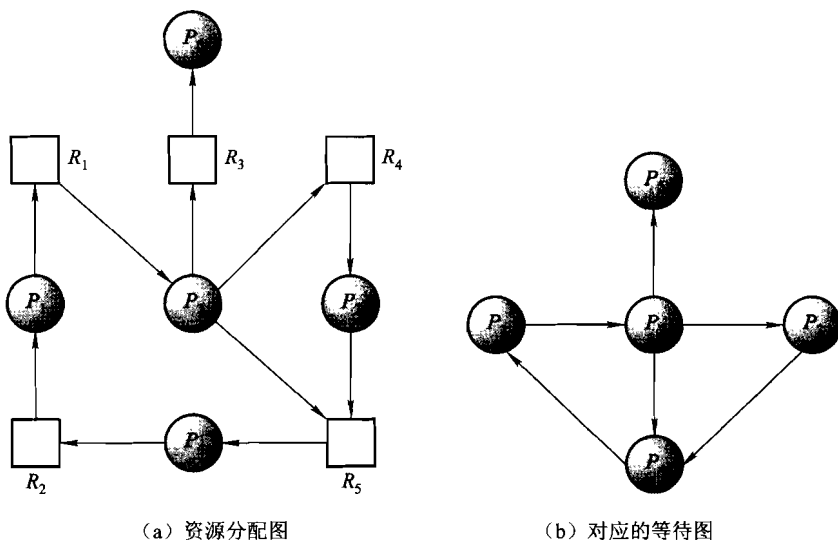


图 7.8 资源分配图和对应的等待图

与以前一样,当且仅当等待图中有一个环,系统中存在死锁。为了检测死锁,系统需要维护等待图,并周期性地调用在图中进行搜索的算法。从图中检测环的算法需要 n^2 级别操作,其中 n 为图中的节点数。

7.6.2 每种资源类型可有多个实例

等待图方案并不适用于每种资源类型可有多个实例的资源分配系统。下面所描述的死锁检测算法适用于这样的系统。该算法使用了一些随时间而变化的数据结构,与银行家算法(参见 7.5.3 小节)相似。

- **Available:** 长度为 m 的向量,表示各种资源的可用实例。
- **Allocation:** $n \times m$ 矩阵,表示当前各进程的资源分配情况。
- **Request:** $n \times m$ 矩阵,表示当前各进程的资源请求情况。如果 $Request[i][j]=k$,那么 P_i 现在正在请求 k 个资源 R_j 。

两向量间小于等于关系与 7.5.3 小节所定义的一样。为了简化起见,将 *Allocation* 和 *Request* 的行作为向量,且分别称为 $Allocation_i$ 和 $Request_i$ 。这里所描述的检测算法为需要完成的所有进程研究各种可能的分配序列。请将本算法与银行家算法做一比较。

① 设 *Work* 和 *Finish* 分别为长度为 m 和 n 的向量。初始化 $Work=Available$ 。对 $i=0,1,\dots,n-1$,如果 $Allocation_i$ 不为 0,则 $Finish[i]=false$; 否则, $Finish[i]=true$ 。

② 找这样的 i , 以便同时使

a. $Finish[i]=false$

b. $Request_i \leq Work$

如果没有这样的 i , 则转到第④步。

③ $Work=Work+Allocation_i$

$Finish[i]=true$

转到第②步。

④ 如果对某个 i ($0 \leq i < n$), $Finish[i]=false$, 则系统处于死锁状态。而且, 如果 $Finish[i]=false$ 。则进程 P_i 死锁。

该算法需要 $m \times n^2$ 级操作来检测系统是否处于死锁状态。

你可能不明白为什么只要确定 $Request_i \leq Work$ (第②步的 b), 就收回了进程 P_i 的资源 (第③步)。已知 P_i 现在不参与死锁 (因 $Request_i \leq Work$), 因此, 可以乐观地认为 P_i 不再需要更多资源以完成其任务, 它会返回其现已分配的所有资源。如果假定的不正确, 那么稍后会发生死锁。下次调用死锁算法时, 就会检测到死锁状态。

为了举例说明这一算法, 考虑这样一个系统, 它有 5 个进程 $P_0 \sim P_4$ 和 3 个资源类型 A 、 B 、 C 。资源类型 A 有 7 个实例, 资源类型 B 有 2 个实例, 资源类型 C 有 6 个实例。假定在时刻 T_0 , 有如下资源分配状态:

	<u>Allocation</u>			<u>Request</u>			<u>Available</u>		
	<i>A</i>	<i>B</i>	<i>C</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>A</i>	<i>B</i>	<i>C</i>
P_0	0	1	0	0	0	0	0	0	0
P_1	2	0	0	2	0	2			
P_2	3	0	3	0	0	0			
P_3	2	1	1	1	0	0			
P_4	0	0	2	0	0	2			

认为系统现不处于死锁状态。事实上, 如果执行检测算法, 会找到这样一个序列 $\langle P_0, P_2, P_3, P_1, P_4 \rangle$ 会导致对所有 i , $Finish[i] = \text{true}$ 。

现在假定进程 P_2 又请求了资源类型 C 的一个实例。这样, $Request$ 矩阵修改成如下形式。

	<u>Request</u>		
	<i>A</i>	<i>B</i>	<i>C</i>
P_0	0	0	0
P_1	2	0	2
P_2	0	0	1
P_3	1	0	0
P_4	0	0	2

认为现在系统是死锁。虽然可收回进程 P_0 所占有的资源, 但是现有资源并不足以满足其他进程的请求。因此, 会存在一个包含进程 P_1, P_2, P_3 和 P_4 的死锁。

7.6.3 应用检测算法

应该何时调用检测算法? 答案取决于两个因素。

- 死锁可能发生的频率是多少?
- 当死锁发生时, 有多少进程会受影响?

如果经常发生死锁, 那么就应经常调用检测算法。分配给死锁进程的资源会一直空着, 直到死锁被打破。另外, 参与死锁循环的进程数量可能会不断增加。

只有当某个进程提出请求且得不到满足时, 才会出现死锁。这一请求可能是完成等待进程链的最后请求。在极端情况下, 每次请求分配不能立即允许时, 就调用死锁检测算法。在这种情况下, 不仅能确定哪些进程处于死锁, 而且也能确定哪个特定进程“造成”了死锁(而实际上, 每个死锁进程都是资源图的环的一个链节, 因此, 其实是所有进程一起造成了死锁)。如果有许多不同资源类型, 那么一个请求可能造成资源图的许多环, 每个环由最近请求所完成且由可确定的进程所“造成”。

当然, 对于每个请求都调用死锁检测算法会引起相当大的计算开销。另一个不太昂贵

的方法是以一个不太高的频率调用检测算法，如每小时一次，或当 CPU 使用率低于 40% 时（死锁最终会使系统性能下降，并造成 CPU 使用率下降）。如果在不确定的时间点调用检测算法，那么资源图会有许多环。通常不能确定死锁进程中是哪些“造成”了死锁。

7.7 死锁恢复

当死锁检测算法确定死锁已存在，那么可以采取多种措施。一种措施是通知操作员死锁已发生，以便操作人员人工处理死锁。另一种措施是让系统从死锁状态中自动恢复过来。打破死锁有两个方法。一个方法是简单地终止一个或多个进程以打破循环等待。另一个方法是从一个或多个死锁进程那里抢占一个或多个资源。

7.7.1 进程终止

有两种方法通过终止进程以取消死锁。不管采用哪个方法，系统都会收回分配给被终止进程的所有资源。

- **终止所有死锁进程。**这种方法显然终止了死锁循环，但其代价也大。这些进程可能已计算了很长时间，这些部分计算结果必须放弃，以后可能还要重新计算。

- **一次只终止一个进程直到取消死锁循环为止。**这种方法的开销会相当大，这是因为每次终止一个进程，都必须调用死锁检测算法以确定进程是否仍处于死锁。

终止一个进程并不容易。如果进程正在更新文件，那么终止它会使文件处于不一致状态。类似地，如果进程正在打印文件，那么系统必须将打印机重新设置到正确状态，以便打印下一个文件。

如果采用了部分终止，那么对于给定死锁进程，必须确定终止哪个进程或哪些进程可以打破死锁。这个确定类似于 CPU 调度问题，是个策略选择。该问题基本上是个经济问题，应该终止代价最小的进程。然而，“代价最小”并不精确，许多因素都影响着应选择哪个进程，包括：

- ① 进程的优先级是什么？
- ② 进程已计算了多久，进程在完成指定任务之前还需要多久？
- ③ 进程使用了多少什么类型的资源（例如，这些资源是否容易抢占）？
- ④ 进程需要多少资源以完成？
- ⑤ 多少进程需要被终止？
- ⑥ 进程是交互的还是批处理的？

7.7.2 资源抢占

通过抢占资源以取消死锁，逐步从进程中抢占资源给其他进程使用，直到死锁环被打

破为止。

如果要求使用抢占来处理死锁，那么有三个问题需要处理：

① **选择一个牺牲品**：抢占哪些资源和哪个进程？与进程取消一样，必须确定抢占顺序以使代价最小化。代价因素包括许多参数，如死锁进程所拥有的资源数量，死锁进程到现在为止在其执行过程中所消耗的时间。

② **回滚**：如果从一个进程那里抢占一个资源，那么应对该进程做些什么安排？显然，该进程不能正常执行，它缺少所需要的资源。必须将进程回滚到某个安全状态，以便从该状态重启进程。通常确定一个安全状态并不容易，所以最简单的方法是完全回滚：终止进程并重新执行。更为有效的方法是将进程回滚到足够打破死锁。另一方面，这种方法要求系统维护有关运行进程状态的更多信息。

③ **饥饿**：如何确保不会发生饥饿？即如何保证资源不会总是从同一个进程中被抢占？如果一个系统是基于代价来选择牺牲进程，那么同一进程可能总是被选为牺牲品。结果，这个进程永远不能完成其指定任务，任何实际系统都需要处理这种饥饿情况。显然，必须确保一个进程只能有限地被选择为牺牲品。最为常用的方法是在代价因素中加上回滚次数。

7.8 小 结

如果两个或更多的进程永久等待某个事件，而该事件只能由这些等待进程的某一个引起，那么会出现死锁状态。从原理上来说，有三种方法可以处理死锁：

- 使用一些协议来预防或避免死锁，确保系统永远都不会进入死锁状态。
- 允许系统进入死锁状态，检测死锁，并恢复。
- 忽略这个问题，并假设系统中永远都不会出现死锁。

第三种方法为绝大多数的系统所采用，包括 UNIX 和 Windows。

当且仅当系统内的 4 个必要条件同时成立时（互斥、占有并等待、非抢占、循环等待），才会发生死锁。为了预防死锁，要确保这 4 个必要条件中的至少一个不成立。

死锁避免算法要比预防算法要求低，只要事先了解进程使用资源的情况即可。例如，银行家算法需要知道每个进程所请求的每种资源的最大数量。采用这种信息，可以采用死锁避免算法。

如果不采用协议以确保死锁不会发生，那么就必须使用检测并恢复方案。必须调用检测算法以确定是否出现了死锁。如果检测到死锁，那么系统必须通过终止某些死锁进程，或通过抢占某些死锁进程的资源从死锁中恢复。

如通过抢占来处理死锁，那么有三点必须要考虑：选择一个牺牲品、回滚以及饥饿。如果系统主要根据代价以选择牺牲进程来进行回滚，那么可能会出现饥饿现象，导致所选

择进程永远不能完成指定任务。

最后，在这些方法有没有一个可独自处理操作系统的所有资源分配问题上，研究人员有些争议。不过，通过这些方法组合起来，可以最佳地处理系统的各种类型资源的分配问题。

习 题

7.1 假设有如图 7.9 所示的交通死锁情况：

- 证明这个例子中实际上包括了死锁发生的 4 个必要条件。
- 给出一个简单的规则使这个系统避免死锁。

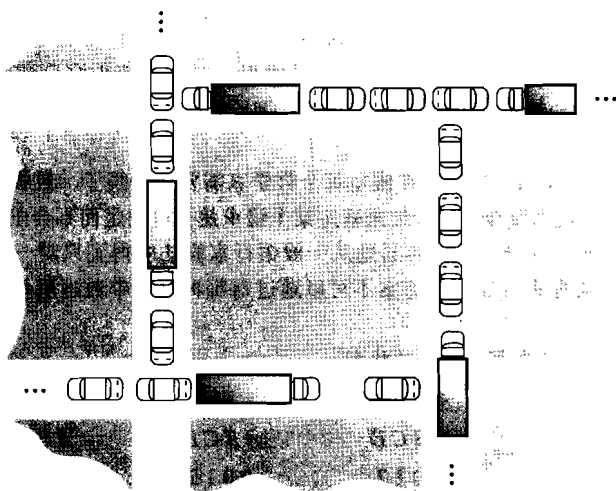


图 7.9 习题 7.1 交通死锁

7.2 当哲学家一次只拿一支筷子时，哲学家就餐问题会出现死锁。请讨论这种情况下的 4 个死锁必要条件确实存在，并讨论如何通过取消 4 个中的一个必要条件来避免死锁。

7.3 一种防止死锁的方法是有个更高优先级的资源，在申请其他资源前，必须先申请该高优先级资源。例如，如果多个线程试图访问同步对象 A 、 $\dots$ 、 E ，可能出现死锁（这些同步对象可以包括互斥、信号量、条件变量等）。通过增加第 6 个对象 F ，可以防止死锁。当任一线程需要获取对象 A 、 $\dots$ 、 E 的同步锁时，它必须先获取对象 F 的锁。这种方法称为包含：对象 A 、 $\dots$ 、 E 的锁包含在对象 F 的锁中。请将这一方法与 7.4.4 小节中的循环等待做一比较。

7.4 根据如下两点，比较循环等待方法与各种死锁避免方法（如银行家算法）：

- 运行时开销。
- 系统吞吐量。

7.5 在一个真实的计算机系统中，可用的资源和进程对资源的要求都不会持续很久（几个月）。资源

会损坏和被替换, 新的进程会进入和离开系统, 新的资源会被购买和加入系统。如果用银行家算法控制死锁, 下面哪些变化在什么情况下是安全的 (不会导致死锁)?

- a. 增加可用资源 (新的资源被加入系统)
- b. 减少可用资源 (资源被从系统中永久性地移出)
- c. 增加一个进程的 *Max* (进程需要更多的资源, 超过所允许的资源)
- d. 减少一个进程的 *Max* (进程不再需要那么多资源)
- e. 增加进程的数量
- f. 减少进程的数量

7.6 假设有 4 个相同类型的资源被 3 个进程共享。每个进程最多需要 2 个资源。试证明这个系统不会死锁。

7.7 假设一个系统有 m 个相同类型的资源被 n 个进程共享, 进程每次只请求或释放一个资源。试证明只要符合下面两个条件, 系统就不会发生死锁。

- a. 每个进程需要资源的最大值在 $1 \sim m$ 之间。
- b. 所有进程需要资源的最大值的和小于 $m+n$ 。

7.8 考虑这样的哲学家就餐问题: 筷子放在桌子中央, 且一个哲学家可使用任两只筷子。假定一次只能请求一只筷子。设计一简单规则, 可以根据现有筷子分配来确定某个请求是否可满足而不会出现死锁。

7.9 考虑与前面问题相同的情景。现在假定每个哲学家需要三只筷子, 且资源请求必须仍然单独进行。设计一简单规则, 可以根据现有筷子分配来确定某个请求是否可满足而不会出现死锁。

7.10 将数组的维数减少到 1, 可以容易地从一般银行家算法中得到只对一种类型资源的银行家算法。给出例子说明多资源类型银行家算法方案不可以通过对每种资源单独运用单资源银行家算法的方法来实现。

7.11 考虑下面的一个系统在某一时刻的状态。

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	<i>A B C D</i>	<i>A B C D</i>	<i>A B C D</i>
P_0	0 0 1 2	0 0 1 2	1 5 2 0
P_1	1 0 0 0	1 7 5 0	
P_2	1 3 5 4	2 3 5 6	
P_3	0 6 3 2	0 6 5 2	
P_4	0 0 1 4	0 6 5 6	

使用银行家算法回答下面问题:

- a. *Need* 矩阵的内容是怎样的?
- b. 系统是否处于安全状态?
- c. 如果从进程 P_1 发来一个请求 (0,4,2,0), 这个请求能否立刻被满足?

7.12 死锁检测算法的最乐观的假设是什么? 这个假设为什么不成立?

7.13 编写一个多线程程序, 实现 7.5.3 小节所描述的银行家算法。创建 n 个线程来向银行申请或释放资源。只有保证系统安全, 银行家才会批准请求。可以用 Pthread 或 Win32 线程来编程。注意共享数据的并发访问要安全。可以通过互斥锁 (Pthread 或 Win32 API 中都有) 来保证访问安全。关于这两个库的互斥锁的讨论, 请参见第 6 章的“生产者-消费者问题”项目。

7.14 有一个单道的桥，连接两个 Vermont 村庄：北 Tunbridge 和南 Tunbridge。这个村庄的农场主通过这个桥，将其收成送到附近的城镇。如果北 Tunbridge 和南 Tunbridge 的农场主同时使用这个桥，那么会出现死锁现象（Vermont 农场主比较顽固，不愿后退）。通过信号量，设计一算法以防止死锁。开始时，不必考虑饥饿问题（如向北的农场主通过使用桥不让向南的农场主使用，或相反）。

7.15 修改习题 7.14 的解决方案，以便它不会出现饥饿问题。

文献注记

Dijkstra[1965a]是死锁领域最早和最有影响的贡献者。Holt[1972]是本章中所展示的将死锁问题用图理论模型进行形式化的第一人。Holt[1972]论述了饥饿问题。Hyman[1985]给出了堪萨斯州议会死锁的例子。最近的有关死锁的研究，见 Levine [2003]。

Havender[1968]曾设计了 IBM OS/360 系统中的资源排序方案，给出了多种死锁预防算法。

Dijkstra[1965]提出了对单个资源类型的银行家死锁避免算法。Habermann[1969]将此算法扩展到多种资源。习题 7.6 和 7.7 来自 Holt[1971]。

Coffman 等[1971]给出了 7.6.2 小节中描述的单种资源多实例的死锁检测算法。

Bach[1987]叙述了在传统的 UNIX 内核中有多少种死锁处理方法。Culler [1998]、Rodeheffer 与 Schroeder [1991]讨论了网络死锁问题的解决方案。

Baldwin [2002]给出了证人锁顺序校检器。

第三部分 内存管理

计算机系统的主要用途是执行程序。在执行时，这些程序及其所访问的数据必须在内存里（至少部分是如此）。

为改善 CPU 的使用率和对用户的响应速度，计算机必须在内存里保留多个进程。内存管理方案有很多，以适应各种不同的需求，每个算法的有效性与特定情况有关。对系统内存管理方案的选择取决于很多因素，特别是系统的硬件设计。每个算法都需要有自己的硬件支持。

第8章 内存管理

在第5章，讨论了CPU如何被一组进程所共享。正是由于CPU调度的结果，才能提高CPU的使用率和计算机对用户的响应速度。但是，为了实现这一性能改进，必须将多个进程保存在内存中；也就是说，必须共享内存。

本章将讨论各种内存管理的方法。内存管理算法有很多，从简单的裸机方法，到分页和分段策略，各种方法都有其优点和缺点。为特定系统选择内存管理方法取决于很多因素，特别是系统的硬件设计。正如将会看到的，尽管现在的设计已经将硬件和操作系统紧密地结合在一起，但是许多算法仍然需要硬件的支持。

本章目标

- 详细描述内存硬件的各种组织方法。
- 讨论各种内存管理技术，包括分页与分段。
- 详细描述 Intel Pentium 芯片，它支持纯分段和带分页的分段。

8.1 背景

正如第1章所述，内存是现代计算机运行的中心。内存由很大一组字或字节组成，每个字或字节都有它们自己的地址。CPU根据程序计数器（PC）的值从内存中提取指令，这些指令可能会引起进一步对特定内存地址的读取和写入。

例如，一个典型指令执行周期，首先从内存中读取指令。接着该指令被解码，且可能需要从内存中读取操作数。在指令对操作数执行后，其结果可能被存回到内存。内存单元只看到地址流，而并不知道这些地址是如何产生的（由指令计数器、索引、间接寻址、实地址等）或它们是什么地址（指令或数据）。相应地，可以忽略内存地址是如何由程序产生的，而只是对由运行中的程序产生的内存地址感兴趣。

这部分首先讨论与内存管理技术有关的几个问题，包括基本硬件概述，符号内存地址到实际物理地址的绑定，以及逻辑地址与物理地址的差别。最后讨论动态装载、动态链接代码及共享库。

8.1.1 基本硬件

CPU所能直接访问的存储器只有内存和处理器内的寄存器。机器指令可以用内存地址

作为参数，而不能用磁盘地址作为参数。因此，执行指令以及指令使用的数据必须在这些直接可访问的存储设备上。如果数据不在内存中，那么在 CPU 使用前必须先把数据移到内存中。

CPU 内置寄存器通常可以在一个 CPU 时钟周期内完成访问。对于寄存器中的内容，绝大多数 CPU 可以在一个时钟周期内解析并执行一个或多个指令，而对于内存（其访问通过内存总线上的事务进行）就不行了。完成内存访问可能需要多个 CPU 时钟周期，由于没有数据以便完成正在执行的指令，CPU 通常需要暂停（stall）。由于内存访问频繁，这种情况是难以忍受的。解决方法是在 CPU 与内存之间，增加高速内存。这种协调速度差异的内存缓存区，称为**高速缓存（cache）**，这在 1.8.3 小节已讨论过了。

除了保证访问物理内存的相对速度之外，还要确保操作系统不被用户进程所访问，以及确保用户进程不被其他用户进程访问。这种保护可通过硬件来实现。硬件实现有许多方法，将在本章后面讨论。这里，只简述一种可能方案。

首先需要确保每个进程都有独立的内存空间。为此，需要确定进程可访问的合法地址的范围，并确保进程只访问其合法地址。如图 8.1 所示，通过两个寄存器即基地址寄存器和界限地址寄存器，可以实现这种保护。**基地址寄存器（base register）**含有最小的合法物理内存地址，而**界限地址寄存器（limit register）**决定了范围的大小。例如，如果基地址寄存器为 300040，而界限寄存器为 120900，那么程序可以合法访问从 300040 到 420940（含）的所有地址。

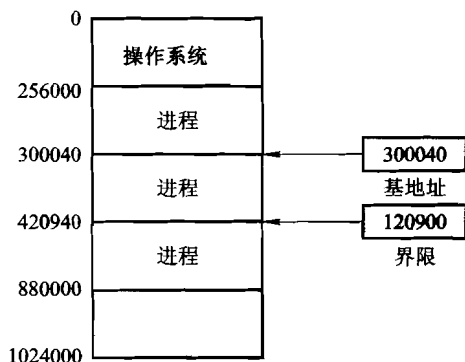


图 8.1 基地址寄存器和界限地址寄存器定义逻辑地址空间

内存空间保护的实现,是通过 CPU 硬件对用户模式所产生的每一个地址与寄存器的地址进行比较来完成的。如用户模式下执行的程序试图访问操作系统内存或其他用户内存,则会陷入到操作系统,并作为致命错误处理(图 8.2)。这种方案防止用户程序(有意或无意地)修改操作系统或其他用户的代码或数据结构。

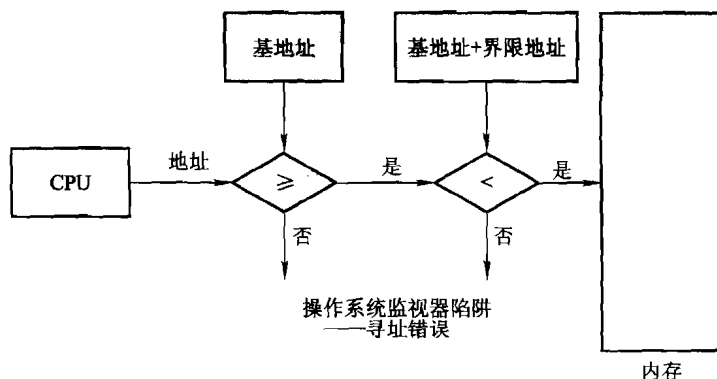


图 8.2 采用基地址寄存器和界限地址寄存器的硬件地址保护

只有操作系统可以通过特殊的特权指令来加载基地址寄存器和界限地址寄存器。由于特权指令只可在内核模式下执行，而只有操作系统在内核模式下执行，所以只有操作系统可以加载基地址寄存器和界限地址寄存器。这种方案允许操作系统修改这两个寄存器的值，而不允许用户程序修改它们。

操作系统在内核模式下执行，可以无限制地访问操作系统和用户的内存。因此操作系统可以将用户程序装入用户内存，在出错时输出这些程序，访问并修改系统调用的参数等。

8.1.2 地址绑定

通常，程序以二进制可执行文件的形式存储在磁盘上。为了执行，程序被调入内存并放在进程空间内。根据所使用的内存管理方案，进程在执行时可以在磁盘和内存之间移动。在磁盘上等待调入内存以便执行的进程形成输入队列（input queue）。

通常的步骤是从输入队列中选取一个进程并装入内存。进程在执行时，会访问内存中的指令和数据。最后，进程终止，其地址空间将被释放。

许多系统允许用户进程放在物理内存的任意位置。因此，虽然计算机的地址空间从 00000 开始，但用户进程的起始地址不必也是 00000。这种组织方式会影响用户程序能够使用的地址空间。在绝大多数情况下，用户程序在执行前，需要经过好几个步骤，其中有的是可选的（参见图 8.3）。在这些步骤中，地址可能有不同的表示形式。源程序中的地址通常是用符号来表示的（如 count）。编译器通常将这些符号地址绑定（bind）在可重定位的地址（如“从本模块开始的第 14 字节”）。链接程序或加载程序再将这些可重定位的地址绑定成绝对地址（如 74014）。每次绑定都是从一个地址空间到另一个地址空间的映射。

通常，将指令与数据绑定到内存地址有以下几种情况：

- 编译时（compile time）：如果在编译时就知道进程将在内存中的驻留地址，那么就

可以生成绝对代码 (absolute code)。例如, 如果事先就知道用户进程驻留在内存地址 R 处, 那么所生成的编译代码就可以从该位置开始并向后扩展。如果将来开始地址发生变化, 那么就必须重新编译代码。MS-DOS 的 .COM 格式程序就是在编译时绑定成绝对代码的。

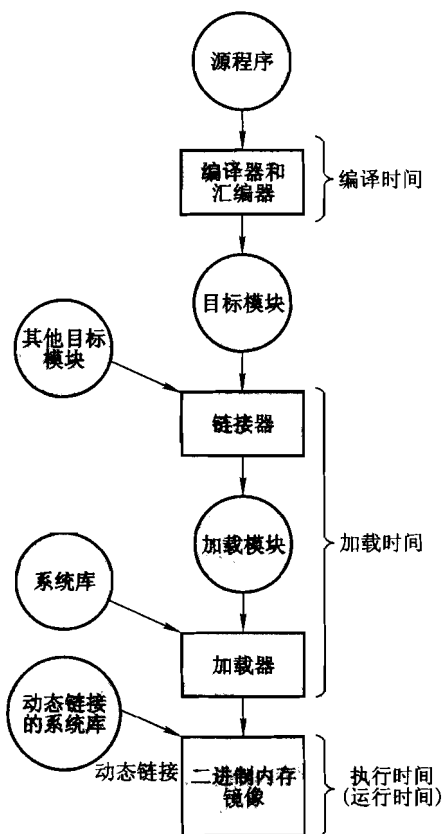


图 8.3 一个用户程序的多步骤处理

- **加载时 (load time)**：如果在编译时并不知道进程将驻留在内存的什么地方, 那么编译器就必须生成可重定位代码 (relocatable code)。对于这种情况, 最后绑定会延迟到加载时才进行。如果开始地址发生变化, 只需重新加载用户代码以引入改变值。

- **执行时 (execution time)**：如果进程在执行时可以从一个内存段移到另一个内存段, 那么绑定必须延迟到执行时才进行。正如 8.1.3 小节所述, 采用这种方案需要特定的硬件。绝大多数通用计算机操作系统采用这种方法。

本章的主要部分将描述如何在计算机系统有效地实现这些绑定, 并将讨论合适的硬件支持。

8.1.3 逻辑地址空间与物理地址空间

CPU 所生成的地址通常称为**逻辑地址** (logical address)，而内存单元所看到的地址 (即加载到**内存地址寄存器** (memory-address register) 中的地址) 通常称为**物理地址** (physical address)。

编译和加载时的地址绑定方法生成相同的逻辑地址和物理地址。但是，执行时的地址绑定方案导致不同的逻辑地址和物理地址。对于这种情况，通常称逻辑地址为**虚拟地址** (virtual address)。在本书中，对**逻辑地址**和**虚拟地址**不作区分。由程序所生成的所有逻辑地址的集合称为**逻辑地址空间** (logical address space)，与这些逻辑地址相对应的所有物理地址的集合称为**物理地址空间** (physical address space)。因此，对于执行时地址绑定方案，逻辑地址空间与物理地址空间是不同的。

运行时从虚拟地址到物理地址的映射是由被称为**内存管理单元** (memory-management unit, MMU) 的硬件设备来完成的。正如将在 8.3 到 8.7 节所要讨论的，有许多可选择的方法来完成这种映射。在此，用一个简单的 MMU 方案来实现这种映射，这是 8.1.1 小节所描述的基地址寄存器方案的推广。基地址寄存器在这里称为**重定位寄存器** (relocation register)。用户进程所生成的地址在送交内存之前，都将加上重定位寄存器的值 (如图 8.4 所示)。例如，如果基地址为 14000，那么用户对位置 0 的访问将动态地重定位为位置 14000；对地址 346 的访问将映射为位置 14346。运行于 Intel 80x86 系列 CPU 的 MS-DOS 操作系统在加载和运行进程时，可使用 4 个重定位地址寄存器。

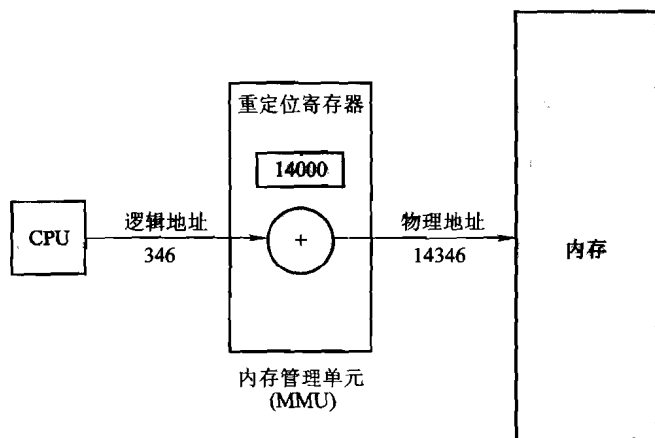


图 8.4 使用重定位寄存器的动态重定位

用户程序决不会看到**真正的**物理地址。程序可以创建一个指向位置 346 的指针，将它保存在内存中，使用它，将它与其他地址进行比较，等等，所有这些操作都是基于 346 进

行的。只有当它作为内存地址时（例如，在间接加载和存储时），它才进行相对于基地址寄存器的重定位。用户程序处理逻辑地址，内存映射硬件将逻辑地址转变为物理地址。这种运行时绑定已在 8.1.2 小节中讨论过。所引用的内存地址只有在引用时才最后定位。

现在有两种不同的地址：逻辑地址（范围为 0 到 max ）和物理地址（范围为 $R+0$ 到 $R+max$ ，其中 R 为基地址）。用户只生成逻辑地址，且认为进程的地址空间为 0 到 max 。用户提供逻辑地址，这些地址在使用前必须映射到物理地址。

*逻辑地址空间绑定到单独的一套物理地址空间*这一概念对内存的管理至关重要。

8.1.4 动态加载

迄今为止所讨论的是一个进程的整个程序和数据必须处于物理内存中，以便执行。因此进程的大小受物理内存大小的限制。为了获得更好的内存空间使用率，可以使用**动态加载**（dynamic loading）。采用动态加载时，一个子程序只有在调用时才被加载。所有子程序都以可重定位的形式保存在磁盘上。主程序装入内存并执行。当一个子程序需要调用另一个子程序时，调用子程序首先检查另一个子程序是否已加载。如果没有，可重定位的链接程序将用来加载所需要的子程序，并更新程序的地址表以反映这一变化。接着，控制传递给新加载的子程序。

动态加载的优点是不用的子程序决不会被加载。如果大多数代码需要用来处理异常情况，如错误处理，那么这种方法特别有用。对于这种情况，虽然总体上程序比较大，但是所使用的部分（即加载的部分）可能小很多。

动态加载不需要操作系统提供特别的支持。利用这种方法来设计程序主要是用户的责任。不过，操作系统可以帮助程序员，如提供子程序库以实现动态加载。

8.1.5 动态链接与共享库

图 8.3 也显示了**动态链接库**（dynamically linked library）。有的操作系统只支持**静态链接**（static linking），此时系统语言库的处理与其他目标模块一样，由加载程序合并到二进制程序镜像中。动态链接的概念与动态加载相似。只是这里不是将加载延迟到运行时，而是将链接延迟到运行时。这一特点通常用于系统库，如语言子程序库。没有这一点，系统上的所有程序都需要一份语言库的副本（或至少那些被程序所引用的子程序）。这一要求浪费了磁盘空间和内存空间。

如果有动态链接，二进制镜像中对每个库程序的引用都有一个**存根**（stub）。存根是一小段代码，用来指出如何定位适当的内存驻留库程序，或如果该程序不在内存时应如何装入库。当执行存根时，它首先检查所需子程序是否已在内存中。如果不在，就将子程序装入内存。不管怎样，存根会用子程序地址来替换自己，并开始执行子程序。因此，下次再执行该子程序代码时，就可以直接进行，而不会因动态链接产生任何开销。采用这种方案，

使用语言库的所有进程只需要一个库代码副本就可以了。

动态链接也可用于库更新（如修改漏洞）。一个库可以被新的版本所替代，且使用该库的所有程序会自动使用新的版本。没有动态链接，所有这些程序必须重新链接以便访问新的库。为了不使程序错用新的、不兼容版本的库，程序和库将包括版本信息。多个版本的库可以都装入内存，程序将通过版本信息来确定使用哪个库副本。不太重要的改动保持了同样的版本号，而重要改动增加版本号。因此，只有用新库编译的程序才会受新库的不兼容变化影响。在新程序装入之前所链接的其他程序可以继续使用老库。这种系统也称为**共享库**。

与动态加载不同，动态链接通常需要操作系统的帮助。如果内存中进程是彼此保护的，那么只有操作系统才可以检查所需子程序是否在其他进程内存空间内，或是允许多个进程访问同一内存地址。在 8.4.4 小节讨论分页时，将进一步进行讨论。

8.2 交 换

进程需要在内存中以便执行。不过，进程可以暂时从内存中**交换（swap）**到**备份存储（backing store）**上，当需要再次执行时再调回到内存中。例如，假如有一个 CPU 调度算法采用轮转法的多道程序环境，当时间片用完，内存管理器开始将刚刚执行过的进程换出，将另一进程换入到刚刚释放的内存空间中（图 8.5）。同时，CPU 调度器可以将时间片分配给其他已在内存中的进程。当每个进程用完时间片，它将与另一进程进行交换。在理想情况下，内存管理器可以以足够快的速度交换进程，以便当 CPU 调度器需要调度 CPU 时，总有进程在内存内可以执行。除此之外，时间片必须足够大，以保证交换之间可以进行一定量的计算。

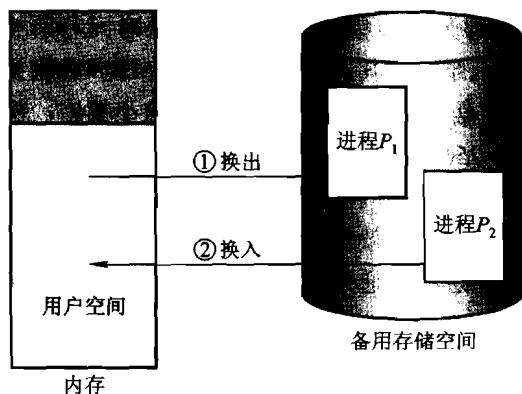


图 8.5 使用磁盘作为备份存储的两个进程的交换

这种交换策略的变种被用在基于优先级的调度算法中。如果有一个更高优先级的进程且需要服务，内存管理器可以交换出低优先级的进程，以便可以装入和执行更高优先级的进程。当更高优先级的进程执行完后，低优先级进程可以交换回内存以继续执行。这种交换有时称为滚出（roll out）和滚入（roll in）。

通常，一个交换出的进程需要交换回它原来所占有的内存空间。这一限制是由地址绑定方式决定的。如果绑定是在汇编时或加载时所定的，那么就不可以移动到不同的位置。如果绑定在运行时才确定，由于物理地址是在运行时才确定的，那么进程可以移到不同的地址空间。

交换需要备份存储。备份存储通常是快速磁盘。这必须足够大，以便容纳所有用户的内存镜像副本，它也必须提供对这些内存镜像的直接访问。系统有一个就绪队列，它包括在备份存储或在内存中准备运行的所有进程。当 CPU 调度程序决定执行进程时，它调用调度程序。调度程序检查队列中的下一进程是否在内存中。如果不在内存中且没有空闲内存空间，调度程序将一个已在内存中的进程交换出去，并换入所需要的进程。然后，它重新装载寄存器，并将控制转交给所选择的进程。

交换系统的上下文切换时间比较长。为了明确上下文切换时间的概念，假设用户进程的大小为 10 MB 且备份存储是传输速度为 40 MBps 的标准硬盘。10 MB 进程传入或传出内存的时间为

$$\begin{aligned} 10\,000\text{ KB}/40\,000\text{ KBps} &= 1/4\text{ s} \\ &= 250\text{ ms} \end{aligned}$$

假定无须磁头寻址且平均延迟为 8 ms，交换时间为 258 ms。由于需要换出和换入，因此总的交换时间约为 516 ms。

为了有效使用 CPU，需要使每个进程的执行时间比交换时间长。例如，对于轮转法 CPU 调度算法，时间片应比 0.516 s 要大。

注意交换时间的主要部分是转移时间。总的转移时间与所交换的内存空间总量成正比。如果有这样一个计算机系统，其内存空间为 512 MB，驻留操作系统为 25 MB，用户的最大空间为 487 MB。但是，许多用户进程可能比这小很多，例如为 10 MB。10 MB 进程的换出需要 258 ms，而 256 MB 的交换需要 6.4s。因此，知道一个用户进程所真正需要的内存空间，而不是其可能所需要的内存空间，是非常有用的。这样，只需要交换真正使用的内存，以减少交换时间。为有效使用这种方法，用户需要告诉系统其内存需求情况。因此，具有动态内存需求的进程要通过系统调用（请求内存和释放内存）来通知操作系统其内存需求变化情况。

交换也受其他因素所限制。如果要换出进程，那么必须确保该进程完全处于空闲状态。尤其值得关注的是待处理 I/O。假如要换出一个进程以释放内存，而该进程正在等待 I/O 操作。如果 I/O 异步访问用户内存的 I/O 缓冲区，那么该进程就不能被换出。假定由于设备

忙, I/O 操作在排队等待。如果换出进程 P_1 而换入进程 P_2 , 那么 I/O 操作可能试图使用现在已属于进程 P_2 的内存。对这个问题有两种解决方法, 一是不能换出有待处理 I/O 的进程, 二是 I/O 操作的执行只能使用操作系统缓冲区。仅当换入进程后, 才执行操作系统缓冲与进程内存之间的数据转移。

前面提到交换时假设不需要或只需要少许移动磁头, 需要进一步解释。有关详细情况, 将在第 12 章涉及外存结构时再讨论。简单地说, 交换空间通常作为磁盘的一整块, 且独立于文件系统, 因此使用就可能很快。

现在, 标准交换使用不多。交换需要很多时间, 而且提供很少的执行时间, 因此这不是一种有效的内存管理解决方案。然而, 一些交换的变种却在许多系统中得以应用。

一种修正过的交换就在许多 UNIX 版本中得以使用。通常并不执行交换, 但当有许多进程运行且内存空间吃紧时, 交换开始启动。如果系统负荷降低, 那么交换就暂停。UNIX 所用的内存管理将在 21.7 节和附录 A.6 中讨论。

早期 PC 缺乏高级硬件 (或者是能充分利用好硬件的操作系统) 来实现高级内存的管理方法, 但是通过使用修正过的交换可以运行多个大进程。一个重要例子就是 Microsoft Windows 3.1 操作系统, 该系统内存中的进程可并发执行。如果需要装入一个进程, 但没有足够的内存空间, 那么一个老的进程就被交换到磁盘上。该操作系统并不支持完全交换, 这是因为不是调度器而是用户来决定何时为一个进程抢占另一进程。换出的进程一直处于换出状态, 直到用户再选择执行该进程为止。后来的 Microsoft 操作系统充分利用了 PC 的高级 MMU 特性。本章的 8.4 节和第 9 章 (讨论虚拟内存) 将探索这些特性。

8.3 连续内存分配

内存必须容纳操作系统和各种用户进程, 因此应该尽可能有效地分配内存的各个部分。本节将介绍一种常用方法——连续内存分配。

内存通常分为两个区域: 一个用于驻留操作系统, 另一个用于用户进程。操作系统可以位于低内存, 也可位于高内存。影响这一决定的主要因素是中断向量的位置。由于中断向量通常位于低内存, 因此程序员通常将操作系统也放在低内存。在本书中, 只讨论操作系统位于低内存的情况。其他情况的讨论类似。

通常需要将多个进程同时放在内存中, 因此需要考虑如何为输入队列中需要调入内存的进程分配内存空间。采用连续内存分配 (contiguous memory allocation) 时, 每个进程位于一个连续的内存区域。

8.3.1 内存映射与保护

在讨论内存分配前, 必须先讨论内存映射与保护问题。通过采用重定位寄存器 (已在

8.1.3 小节讨论)和界限地址寄存器(已在 8.1.1 小节讨论),可以实现这种保护。重定位寄存器含有最小的物理地址值;界限地址寄存器含有逻辑地址的范围值(例如,重定位=100040,界限=74600)。有了重定位寄存器和界限地址寄存器,每个逻辑地址必须小于界限地址寄存器。MMU 动态地将逻辑地址加上重定位寄存器的值后映射成物理地址。映射后的物理地址再送交内存单元(见图 8.6)。

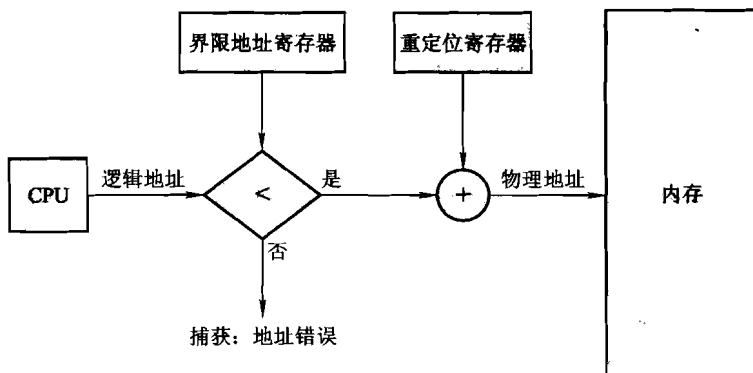


图 8.6 重定位寄存器和界限地址寄存器的硬件支持

当 CPU 调度器选择一个进程来执行时,作为上下文切换工作的一部分,调度程序会用正确的值来初始化重定位寄存器和界限地址寄存器。由于 CPU 所产生的每一地址都需要与寄存器进行核对,所以可以保证操作系统和其他用户程序和数据不受该进程的运行所影响。

重定位寄存器机制为允许操作系统动态改变提供了一个有效方法。许多情况都需要这一灵活性。例如,操作系统的驱动程序需要代码和缓冲空间。如果某驱动程序(或其他操作系统服务)不常使用,可以不必在内存中保留该代码和数据,这部分空间可以用于其他目的。这类代码有时称为暂时(transient)操作系统代码;它们根据需要调入或调出。因此,使用这种代码可以在程序执行时动态改变操作系统的大小。

8.3.2 内存分配

现在来讨论内存分配。最为简单的内存分配方法之一就是将内存分为多个固定大小的分区(partition)。每个分区只能容纳一个进程。因此,多道程序的程度会受分区数所限制。如果使用这种多分区方法(multiple-partition method),当一个分区空闲时,可以从输入队列中选择一个进程,以调入到空闲分区。当进程终止时,其分区可以被其他进程所使用。这种方法最初为 IBM OS/360 操作系统(称为 MFT)所使用,现在已不再使用。下面所描述的方法是固定分区方案的推广(称为 MVT),它主要用于批处理环境。这里所描述的许多思想也可用于采用纯分段内存管理的分时操作系统(参见 8.6 节)。

在可变分区 (variable-partition) 方案里, 操作系统有一个表, 用于记录哪些内存可用和哪些内存已被占用。一开始, 所有内存都可用于用户进程, 因此可以作为一大块可用内存, 称为孔 (hole)。当有新进程需要内存时, 为该进程查找足够大的孔。如果找到, 可以从该孔为该进程分配所需的内存, 孔内未分配的内存可以下次再用。

随着进程进入系统, 它们将被加入到输入队列。操作系统根据所有进程的内存需要和现有可用内存情况来决定哪些进程可分配内存。当进程分配到空间时, 它就装入内存, 并开始竞争 CPU。当进程终止时, 它将释放内存, 该内存可以被操作系统分配给输入队列中的其他进程。

在任意时候, 有一组可用孔 (块) 大小列表和输入队列。操作系统根据调度算法来对输入队列进行排序。内存不断地分配给进程, 直到下一个进程的内存需求不能满足为止, 这时没有足够大的可用孔来装入进程。操作系统可以等到有足够大的空间, 或者往下扫描输入队列以确定是否有其他内存需求较小的进程可以被满足。

通常, 一组不同大小的孔分散在内存中。当新进程需要内存时, 系统为该进程查找足够大的孔。如果孔太大, 那么就分为两块: 一块分配给新进程, 另一块还回到孔集合。当进程终止时, 它将释放其内存, 该内存将还给孔集合。如果新孔与其他孔相邻, 那么将这些孔合并成大孔。这时, 系统可以检查是否有进程在等待内存空间, 新合并的内存空间是否满足等待进程。

这种方法是通用动态存储分配问题的一种情况 (根据一组空闲孔来分配大小为 n 的请求)。这个问题有许多解决方法。从一组可用孔中选择一个空闲孔的最为常用方法有首次适应 (first-fit)、最佳适应 (best-fit)、最差适应 (worst-fit)。

- **首次适应:** 分配第一个足够大的孔。查找可以从头开始, 也可以从上次首次适应结束时开始。一旦找到足够大的空闲孔, 就可以停止。

- **最佳适应:** 分配最小的足够大的孔。必须查找整个列表, 除非列表按大小排序。这种方法可以产生最小剩余孔。

- **最差适应:** 分配最大的孔。同样, 必须查找整个列表, 除非列表按大小排序。这种方法可以产生最大剩余孔, 该孔可能比最佳适应方法产生的较小剩余孔更为有用。

模拟结果显示首次适应和最佳适应方法在执行时间和利用空间方面都好于最差适应方法。首次适应和最佳适应方法在利用空间方面难分伯仲, 但是首次适应方法要更快些。

8.3.3 碎片

首次适应方法和最佳适应方法算法都有外部碎片问题 (external fragmentation)。随着进程装入和移出内存, 空闲内存空间被分为小片段。当所有总的可用内存之和可以满足请求, 但并不连续时, 这就出现了外部碎片问题, 该问题可能很严重。在最坏情况下, 每两个进程之间就有空闲块 (或浪费)。如果这些内存是一整块, 那么就可以再运行多个进程。

在首次适应和最佳适应之间的选择可能会影响碎片的量（对一些系统来说首次适应更好，对另一些系统最佳适应更好）。另一个影响因素是从空闲块的哪端开始分配（顶端还是末端）。不管使用哪种算法，外部碎片始终是个问题。

根据内存空间总的大小和平均进程大小的不同，外部碎片的重要程度也不同。例如，对采用首次适应方法的统计说明，不管使用什么优化，假定有 N 个可分配块，那么可能有 $0.5N$ 个块为外部碎片。即 $1/3$ 的内存可能不能使用。这一特性称为 **50%规则**。

内存碎片可以是内部的，也可以是外部的。设想有一个 18 464 B 大小的孔，并采用多分区分配方案。假如有一个进程需要 18 462 B。如果只准确分配所要求的块，那么还剩下一个 2 B 的孔。维护这一小孔的开销要比孔本身大得多。因此，通常将内存以固定大小的块为单元（而不是字节）来分配。采用这种方案，进程所分配的内存可能比所要的要大。这两个数字之差称为**内部碎片**，这部分内存存在分区内，但又不能使用。

一种解决外部碎片问题的方法是**紧缩（compaction）**。紧缩的目的是移动内存内容，以便所有空闲空间合并成一整块。但紧缩并非总是可能的。如果重定位是静态的，并且在汇编时或装入时进行的，那么就不能紧缩。紧缩仅在重定位是动态并在运行时可采用。如果地址被动态重定位，可以首先移动程序和数据，然后再根据新基地址的值来改变基地址寄存器。如果能采用紧缩，还需要评估其开销。最简单的合并算法是简单地将所有进程移到内存的一端，而将所有的孔移到内存的另一端，以生成一个大的空闲块。这种方案开销较大。

另一种可能解决外部碎片问题的方法是允许物理地址空间为非连续，这样只要有物理内存就可为进程分配。这种方案有两种互补的实现技术：分页（见 8.4 节）和分段（见 8.6 节）。这种两种技术也可合并（见 8.7 节）。

8.4 分 页

分页（paging）内存管理方案允许进程的物理地址空间可以是非连续的。分页避免了将不同大小的内存块匹配到交换空间上这样的麻烦，前面所述内存管理方案都有这个问题。当位于内存中的代码或数据需要换出时，必须先先在备份存储上找到空间，这时问题就产生了。备份存储也有前面所述与内存相关的碎片问题，只不过访问更慢，因此不适宜采用合并。各种形式的分页由于其优越性，因此通常为绝大多数操作系统所采用。

传统上，分页支持一直是由硬件来处理的。然而，最近的设计是通过将硬件和操作系统相配合来实现分页（尤其是在 64 位微处理器上）。

8.4.1 基本方法

实现分页的基本方法涉及将物理内存分为固定大小的块，称为**帧（frame）**；而将逻辑

内存也分为同样大小的块，称为页（page）。当需要执行进程时，其页从备份存储中调入到可用的内存帧中。备份存储也分为固定大小的块，其大小与内存帧一样。

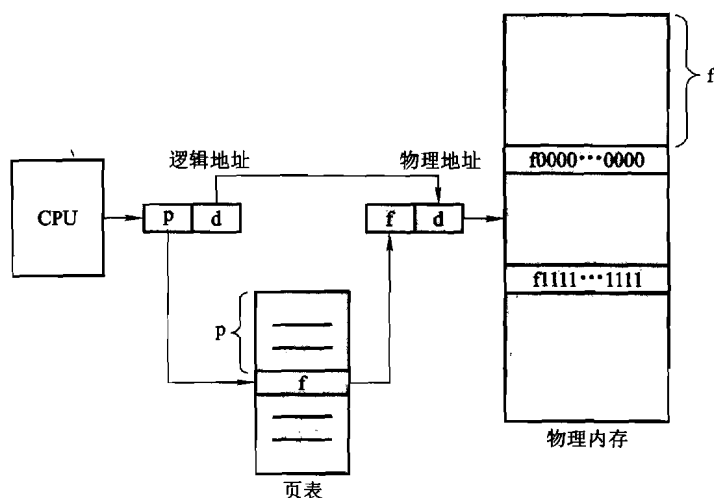


图 8.7 分页的硬件支持

分页硬件支持如图 8.7 所示。由 CPU 生成的每个地址分为两个部分：页号（ p ）和页偏移（ d ）。页号作为页表中的索引。页表包含每页所在物理内存的基地址，这些基地址与页偏移的组合就形成了物理地址，就可送交物理单元。内存的页模型如图 8.8 所示。

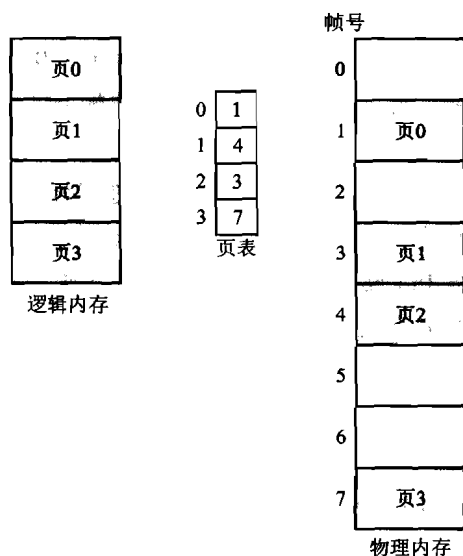
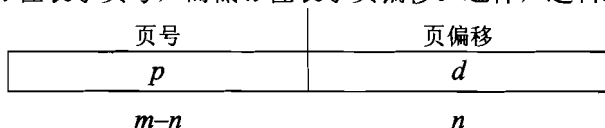


图 8.8 逻辑内存和物理内存的分页模型

页大小（与帧大小一样）是由硬件来决定的。页的大小通常为 2 的幂，根据计算机结构的不同，其每页大小从 512 B~16 MB 不等。选择页的大小为 2 的幂可以方便地将逻辑地址转换为页号和页偏移。如果逻辑地址空间为 2^m ，且页大小为 2^n 单元（字节或字），那么逻辑地址的高 $m-n$ 位表示页号，而低 n 位表示页偏移。这样，逻辑地址如下所示：



其中 p 作为页表的索引，而 d 作为页的偏移。

如图 8.9 所示，如果页大小为 4 B，而物理内存为 32 B（8 页），考虑一下用户视角的内存是如何映射到物理内存的。逻辑地址 0 的页号为 0，页偏移为 0。根据页表，可以查到页号 0 对应为帧 5，因此逻辑地址 0 映射为物理地址 20（=（5×4）+0）。逻辑地址 3（页号为 0，页偏移为 3）映射为物理地址 23（=（5×4）+3）。逻辑地址 4 的页号为 1，页偏移为 0，根据页表，页号 1 对应为帧 6，因此，逻辑地址 4 映射为物理地址 24（=（6×4）+0）。逻辑地址 13 映射为物理地址 9。

读者可能注意到分页也是一种动态重定位。每个逻辑地址由分页硬件绑定为一定的物理地址。采用分页类似于使用一组基（重定位）地址寄存器，每个基地址对应着一个内存帧。

采用分页技术不会产生外部碎片：每个帧都可以分配给需要它的进程。不过，分页有内部碎片。注意分配是以帧为单元进行的。如果进程所要求的内存并不是页的整数倍，那么最后一个帧就可能用不完。例如，如果页的大小为 2 048 B，一个大小为 72 776 B 的进程需要 35 个页和 1 086 B。该进程会得到 36 个帧，因此有 2 048-1 086=962 B 的内部碎片。在最坏情况下，一个需要 n 页再加 1 B 的进程，需要分配 $n+1$ 个帧，这样几乎产生一个整个帧的内部碎片。

如果进程大小与页大小无关，那么可以推测每个进程可能有半页的内部碎片。这一结构意味着小一点的页可能好些。不过，由于页表中的每项也有一定的开销，该开销随着页的增大而降低。而且，磁盘 I/O 操作随着传输量的增大会更为有效（见第 12 章）。一般来说，随着时间的推移，页的大小也随着进程、数据和内存的不断增大而增大。现在，页大小通常为 4~8 KB，有的系统可能支持更大的页。有的 CPU 内核可能支持多种页大小。例如，Solaris 根据页所存储的数据，可使用 8 KB 或 4 MB 的大小的页。研究人员正在研究对快速可变的页大小的支持。

每个页表的条目通常为 4B，不过这是可变的。一个 32 位的条目可以指向 2^{32} 个物理帧中的任一个。如果帧为 4 KB，那么具有 4B 条目的系统可以访问 2^{44} B 大小（或 16 TB）的物理内存。

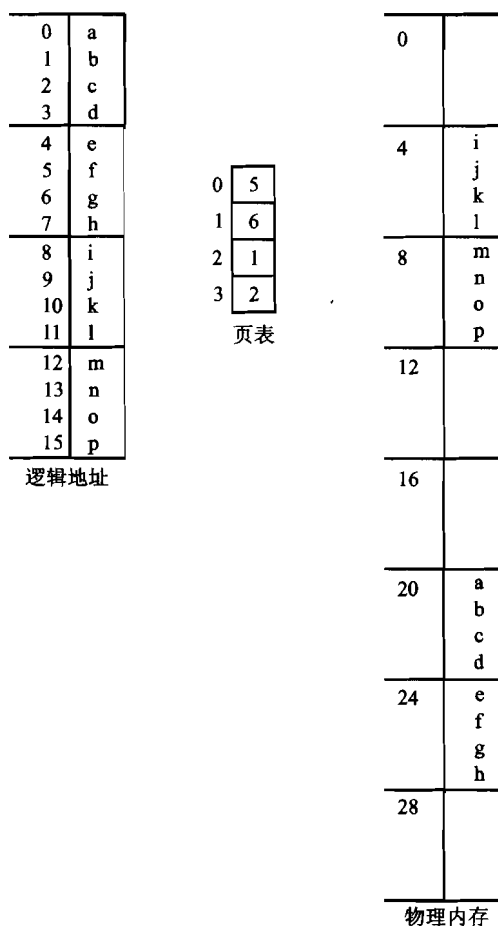


图 8.9 使用 4 B 的页对 32 B 的内存进行分页的例子

当系统进程需要执行时，它将检查该进程的大小（按页来计算），进程的每页都需要一帧。因此，如果进程需要 n 页，那么内存中至少应有 n 个帧。如果有，那么就可分配给新进程。进程的第一页装入一个已分配的帧，帧号放入进程的页表中。下一页分配给另一帧，其帧号也放入进程的页表中等（见图 8.10）。

分页的一个重要特点是用户视角的内存和实际的物理内存的分离。用户程序将内存作为一整块来处理，而且它只包括这一个进程。事实上，一个用户程序与其他程序一起，分布在物理内存上。用户视角的内存和实际的物理内存的差异是通过地址转换硬件协调的。逻辑地址转变成物理地址。这种映射是用户所不知道的，但是受操作系统所控制。注意用户进程根据定义是不能访问非它所占用的内存的。它无法访问其页表所规定之外的内存，

页表只包括进程所拥有的那些页。

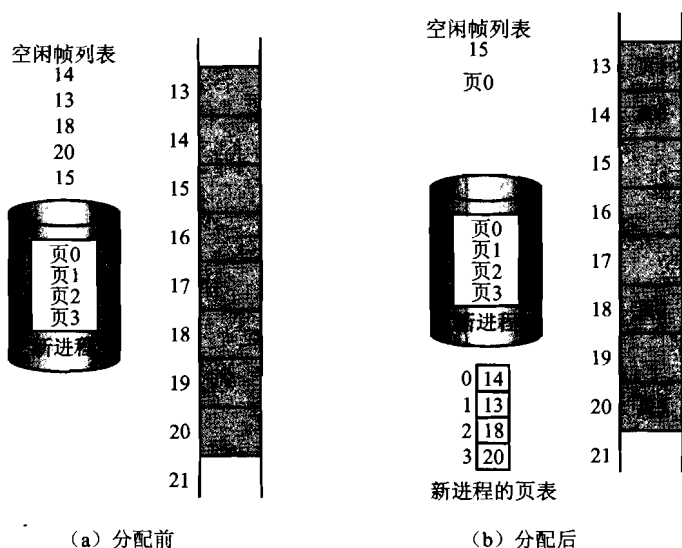


图 8.10 空闲帧

由于操作系统管理物理内存，它必须知道物理内存的分配细节：哪些帧已占用，哪些帧可用，总共有多少帧，等等。这些信息通常保存在称为帧表的数据结构中。在帧表（frame table）中，每个条目对应着一个帧，以表示该帧是空闲还是已占用，如果占用，是被哪个（或哪些）进程的哪个页所占用。

另外，操作系统必须意识到用户进程是在用户空间内执行，且所有逻辑地址必须映射到物理地址。如果用户执行一个系统调用（例如进行 I/O），并提供地址作为参数（如一个缓冲），那么这个地址必须映射成物理地址。操作系统为每个进程维护一个页表的副本，就如同它需要维护指令计数器和寄存器的内容一样。当操作系统必须手工将逻辑地址映射成物理地址时，这个副本可用来将逻辑地址转换为物理地址。当一个进程可分配到 CPU 时，CPU 调度程序可以根据该副本来定义硬件页表。因此，分页增加了切换时间。

8.4.2 硬件支持

每个操作系统都有自己的方法来保存页表。绝大多数都为每个进程分配一个页表。页表的指针与其他寄存器的值（如指令计数器）一起存入进程控制块中。当调度程序需要启动一个进程时，它必须首先装入用户寄存器，并根据所保存的用户页表来定义正确的硬件页表值。

页表的硬件实现有多种方法。最为简单的一种方法是将页表作为一组专用寄存器

(register) 来实现。这些寄存器应用高速逻辑电路来构造, 以便有效地进行分页地址的转换。由于对内存的每次访问都要经过分页表, 因此效率很重要。CPU 调度程序在装入其他寄存器时, 也需要装入这些寄存器。装入或修改页表寄存器的指令是特权级的, 因此只有操作系统才可以修改内存映射图。DEC PDP-11 就是这种类型的结构。它的地址有 16 位, 而页面大小为 8 KB, 因此页表有 8 个条目可放在快速寄存器中。

如果页表比较小 (例如 256 个条目), 那么页表使用寄存器还是比较合理的。但是, 绝大多数当代计算机都允许页表非常大 (如 1 百万个条目)。对于这些机器, 采用快速寄存器来实现页表就不可行了。因而需要将页表放在内存中, 并将**页表基寄存器** (page-table base register, PTBR) 指向页表。改变页表只需要改变这一寄存器就可以, 这也大大降低了切换时间。

采用这种方法的问题是访问用户内存位置需要一些时间。如果要访问位置 i , 那么必须先用 PTBR 中的值再加上页号 i 的偏移, 来查找页表。这一任务需要内存访问。根据所得的帧号, 再加上页偏移, 就得到了真实物理地址。接着就可以访问内存中所需的位置。采用这种方案, 访问一个字节需要两次内存访问 (一次用于页表条目, 一次用于字节)。这样, 内存访问的速度就减半。在绝大多数情况下, 这种延迟是无法忍受的, 还不如采用交换机制。

对这一问题的标准解决方案是采用小但专用且快速的硬件缓冲, 这种缓冲称为**转换表缓冲区** (translation look-aside buffer, TLB)。TLB 是关联的快速内存。TLB 条目由两部分组成: 键 (标签) 和值。当关联内存根据给定值查找时, 它会同时与所有键进行比较。如果找到条目, 那么就得到相应的值域。这种查找方式比较快, 不过硬件也比较昂贵。通常, TLB 的条目数并不多, 通常在 64~1 024 之间。

TLB 与页表一起按如下方法使用: TLB 只包括页表中的一小部分条目。当 CPU 产生逻辑地址后, 其页号提交给 TLB。如果找到页号, 那么也就得到了帧号, 并可用来访问内存。整个任务与不采用内存映射相比, 其时间增加不会超过 10%。

如果页码不在 TLB 中 (称为 **TLB 失效**), 那么就需要访问页表。当得到帧号后, 就可以用它来访问内存 (如图 8.11 所示)。同时, 将页号和帧号增加到 TLB 中, 这样下次再使用时就可很快查找到。如果 TLB 中的条目已满, 那么操作系统会选择一个来替换。替换策略有很多, 从最近最少使用替换 (LRU) 到随机替换等。另外, 有的 TLB 允许有些条目固定下来, 也就是说它们不会从 TLB 中被替换。通常内核代码的条目是固定下来的。

有的 TLB 在每个 TLB 条目中还保存**地址空间标识符** (address-space identifier, ASID)。ASID 可用来唯一地标识进程, 并为进程提供地址空间保护。当 TLB 试图解析虚拟页号时, 它确保当前运行进程的 ASID 与虚拟页相关的 ASID 相匹配。如果不匹配, 那么就作为 TLB 失效。除了提供地址空间保护外, ASID 也允许 TLB 同时包括多个不同进程的条目。如果 TLB 不支持独立的 ASID, 每次选择一个页表时 (例如, 上下文切换时), TLB 就必须被冲

刷 (flushed) 或删除, 以确保下一个进程不会使用错误的地址转换。否则, TLB 中可能有老的条目, 这些条目不包括有效的页号地址, 而包括从上一个进程留下来的无效的物理地址。

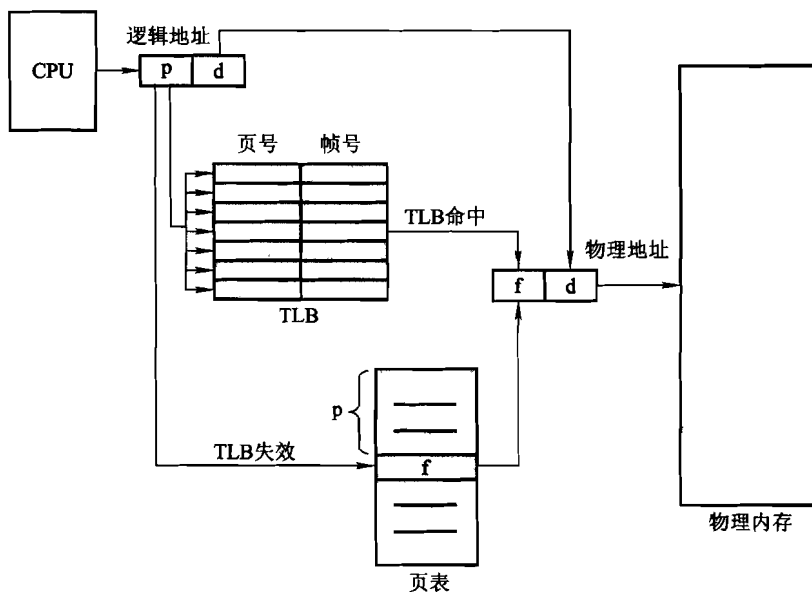


图 8.11 带 TLB 的分页硬件

页号在 TLB 中被查找到的百分比称为**命中率**。80%的命中率意味着有 80%的时间, 可以在 TLB 中找到所需的页号。假如查找 TLB 需要 20 ns, 访问内存需要 100 ns, 如果访问位于 TLB 中的页号, 那么采用内存映射访问需要 120 ns。如果不能在 TLB 中找到, 那么必须先访问位于内存中的页表以得到帧号 (100 ns), 并进而访问内存中的所需字节 (100 ns), 这总共要花费 220 ns。为了得到有效内存访问时间, 必须根据概率来对每种情况进行加权。

$$\text{有效内存访问时间} = 0.80 \times 120 + 0.20 \times 220 = 140 \text{ (ns)}$$

对于这种情况, 现在内存访问速度要慢 40% (从 100~140 ns)。

如果命中率为 98%, 那么

$$\text{有效内存访问时间} = 0.98 \times 120 + 0.02 \times 220 = 122 \text{ (ns)}$$

由于提高了命中率, 现在内存访问速度只慢了 22%。在第 9 章中, 将进一步讨论命中率对 TLB 的影响。

8.4.3 保护

在分页环境下, 内存保护是通过与每个帧相关联的保护位来实现的。通常, 这些位保

存在页表中。

可以用一个位来定义一个页是可读写还是只读的。每次地址引用都要通过页表来查找正确的帧码，在计算物理地址的同时，可以通过检查保护位来验证有没有对只读页进行写操作。对只读页进行写操作会向操作系统产生硬件陷阱（或内存保护冲突）。

可以很容易地扩展这一方法以提供更细致的保护。可以创建硬件以提供只读、读写、只执行保护。或者，通过为每种访问情况提供独立保护位，实现这些访问的各种组合；非法访问会被操作系统捕捉到。

还有一个位通常与页表中的每一条目相关联：**有效-无效位**。当该位为有效时，表示相关的页在进程的逻辑地址空间内，因此是合法（或有效）的页。当该位为无效时，表示相关的页不在进程的逻辑地址空间内。通过使用**有效-无效位**可以捕捉到非法地址。操作系统通过对该位的设置可以允许或不允许对某页的访问。

例如，对于 14 位地址空间（0~16 383）的系统，有一个程序，其有效地址空间为 0~10 468。如果页的大小为 2 KB，那么得到如图 8.12 所示的页表。页 0、1、2、3、4 和 5 的地址可以通过页表正常映射。然而，如果试图产生页 6、7 中的地址，就会发现有效-无效位为无效，这样操作系统就会捕捉到这一非法操作（无效地址引用）。

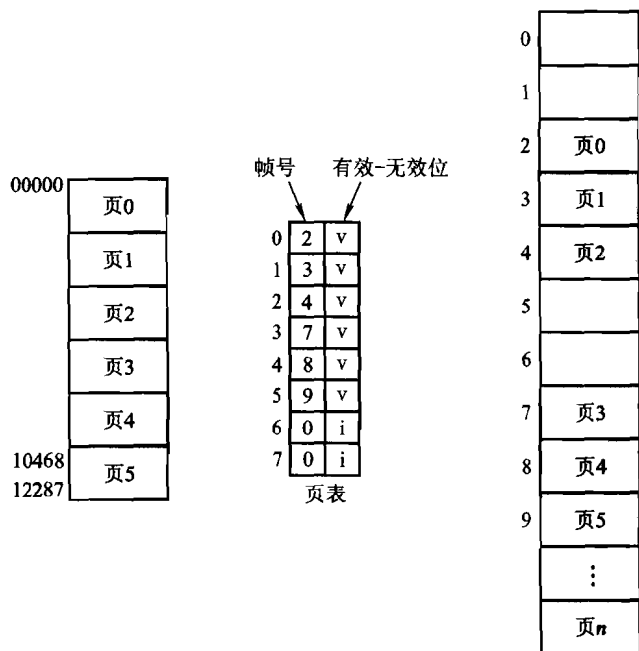


图 8.12 在页表中的有效位 (v) 或无效位 (i)

注意，这种方法也生产了一个问题。由于程序的地址只到 10 468，所以任何超过该地

址的引用都是非法的。不过，由于对页 5 的访问是有效的，因此到 12 287 为止的地址都是有效的，只有 12 288~16 383 的地址才是无效的。这个问题是由于页大小为 2 KB 的原因，也反映了分页的内部碎片。

一个进程很少会使用其所有的地址空间。事实上，许多进程只使用一小部分可用的地址空间。对这些情况，如果为地址范围内的所有页都在页表中建立一个条目，这将是非常浪费的。表中的绝大多数并不会被使用，却占用可用的地址空间。有些系统提供硬件如**页表长度寄存器**(page-table length register, PTLR)来表示页表的大小，该寄存器的值可用于检查每个逻辑地址以验证其是否位于进程的有效范围内。如果检测无法通过，会被操作系统捕捉到。

8.4.4 共享页

分页的优点之一在于可以共享公共代码，这种考虑对分时环境特别重要。考虑一个支持 40 个用户的系统，每个用户都执行一个文本编辑器。如果文本编辑器包括 150 KB 的代码及 50 KB 的数据空间，则需要 8 000 KB 来支持这 40 个用户。如果代码是**可重入代码**(reentrant code，或称为**纯代码**)，则可以共享，如图 8.13 所示。在此将看到 3 个页的编辑器——每页 50 KB 大小（为了简化图，采用了大页面）——在 3 个进程间共享。每个进程都有它自己的数据页。

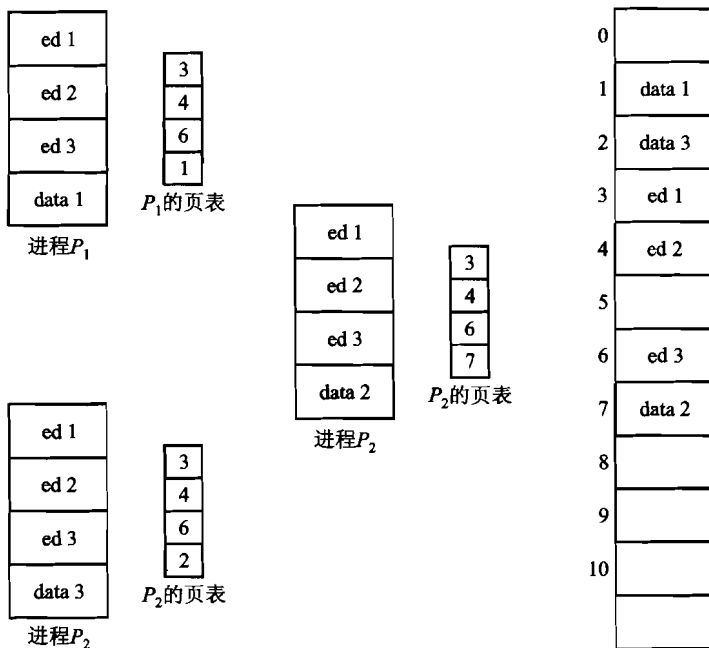


图 8.13 在分页环境下的代码共享

可重入代码是不能自我修改的代码，它从不会在执行期间改变。因此，两个或更多的进程可以在相同的时间执行相同的代码。每个进程都有它自己的寄存器副本和数据存储，以控制进程执行的数据。当然，两个不同的进程的数据也将不同。

只需要在物理内存中保存一个编辑器副本。每个用户的页表映射到编辑器的同一物理副本，而数据页映射到不同的帧。因此，为支持 40 个用户，只需要一个编辑器副本 (150 KB)，再加上 40 个用户数据空间副本 50 KB，总的需求空间为 2 150 KB，而不是 8 000 KB，这是一个明显的节省。

其他常用程序也可能共享，如编译器、窗口系统、运行时库、数据库系统等。要共享，代码必须能重入。共享代码的只读特点不能只通过正确代码来保证，而需要操作系统来强制实现。

一个系统多个进程的内存共享类似于一个任务的多线程地址空间的共享（如第 4 章所述）。第 3 章已描述过共享内存作为一种进程通信机制。有的操作系统通常共享页来实现共享内存。

除了允许多个进程共享同样的物理页外，按页组织内存也提供了许多其他优点。在第 9 章，将讨论其他优点。

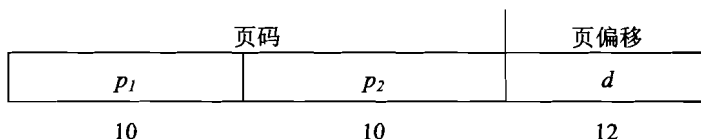
8.5 页表结构

本节将讨论一些组织页表的常用技术。

8.5.1 层次页表

绝大多数现代计算机系统支持大逻辑地址空间 ($2^{32} \sim 2^{64}$)。在这种情况下，页表本身可以非常大。例如，设想一下具有 32 位逻辑地址空间的计算机系统。如果系统的页大小为 4 KB (2^{12})，那么一个页表可以包含 1 百万个条目 ($2^{32}/2^{12}$)。假设每个条目有 4 B，那么每个进程需要 4 MB 物理地址空间来存储页表本身。显然，我们并不可能在内存中连续地分配这个页表。这个问题的一个简单解决方法是将页表划分为更小部分。划分有许多方法。

一种方法是使用两级分页算法，就是将页表再分页（见图 8.14）。仍以之前一个 4 KB 页大小的 32 位系统为例。一个逻辑地址被分为 20 位的页码和 12 位的页偏移。因为要对页表进行再分页，所以该页号可分为 10 位的页码和 10 位的页偏移。这样，一个逻辑地址就分为如下形式：



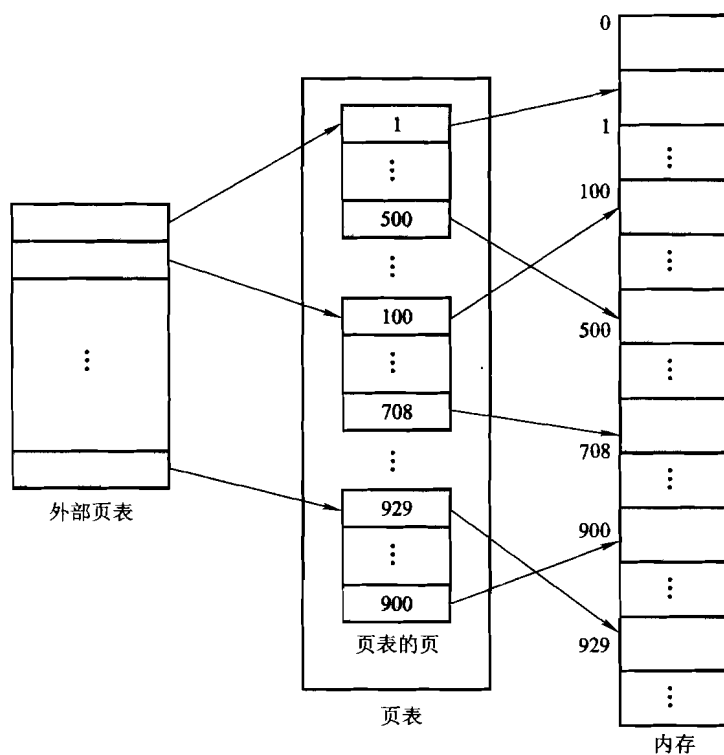


图 8.14 一个两级页表方案

其中, p_1 是用来访问外部页表的索引, 而 p_2 是外部页表的页偏移。采用这种结构的地址转换方法如图 8.15 所示。由于地址转换由外向内, 这种方案也称为向前映射页表 (forward-mapped page table)。

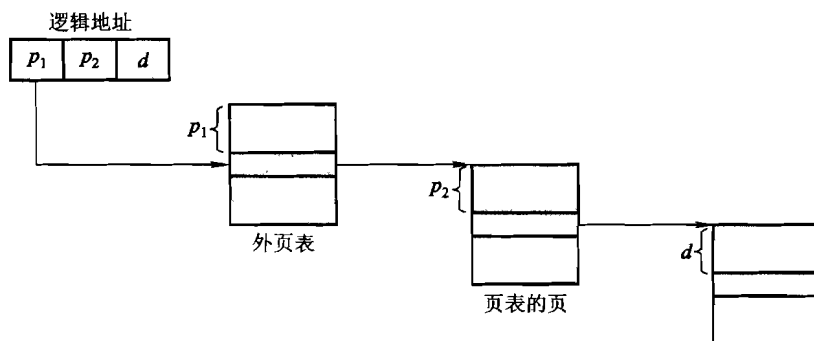


图 8.15 二级 32 位分页体系的地址转换

VAX 体系结构也支持一种两层分页的变种。VAX 是 32 位机器，其页大小为 512B。进程的逻辑地址空间被分为 4 个区，每个区为 2^{30} B。每个区表示一个进程的逻辑地址空间的不同部分。逻辑地址的头两个高位表示适当的区，中间 21 位表示区内的页号，后 9 位表示所需页中的偏移。通过按这种方式来分页，操作系统可以只在进程需要时才使用某些分区。VAX 体系结构的地址按如下形式分为：

区	页	偏移
s	p	d
2	21	9

其中， s 表示区号， p 表示页表的索引，而 d 表示页内的偏移。即使采用此方法，一个 VAX 进程如使用一个区，其一层的页表大小为 $2^{21} \times 4 \text{ B} = 8 \text{ MB}$ 。为了进一步减少对主存的使用，VAX 对用户进程的页表进行换页。

对于 64 位的逻辑地址空间的系统，两级分页方案就不再适合。为了说明这一点，假设系统的页大小为 4 KB(2^{12})。这时，页表可由 2^{52} 条目组成。如果使用两级分页方案，那么内部页表可方便地定为一页长，或包括 2^{10} 个 4 B 的条目。地址形式如下所示：

外部页表	内部页表	偏移
p_1	p_2	d
42	10	12

外部页表有 2^{42} 个条目，或 2^{44} B。避免这样一个大表的显而易见的方法是将外部页表再进一步细分。这种方法也可用于 32 位处理器来增加灵活性和有效性。

可以有很多方式来划分外部页表。可以再分外部页表，得到三层分页方案。假设外部页表由标准大小的页组成 (2^{10} 个条目或者 2^{12} B) 组成，那么 64 位地址空间仍然是很大的：

第二级外部页表	外部页表	Inter page	Offset
p_1	p_2	p_3	d
32	10	10	12

最外部表的大小仍然为 2^{34} B。

下一步是四级分页方案，这时二级外部页表被再次细分。32 位 SPARC 体系结构采用了三级分页方案，而 32 位 Motorola 68030 体系结构采用了四级分页方案。

然而，对于 64 位体系结构，层次页表通常并不适合。例如，64 位 UltraSPARC 体系结构可采用 7 级分页，这可以说是一个有效转换逻辑地址的内存访问的极限。

8.5.2 哈希页表

处理超过 32 位地址空间的常用方法是使用**哈希页表** (hashed page table)，并以虚拟页码作为哈希值。哈希页表的每一条目都包括一个链表的元素，这些元素哈希成同一位置(要

处理碰撞)。每个元素有 3 个域：(1)虚拟页码，(2)所映射的帧号，(3)指向链表中下一个元素的指针。

该算法按如下方式工作：虚拟地址中的虚拟页号转换到哈希表中，用虚拟页号与链表中的每一个元素的第一个域相比较。如果匹配，那么相应的帧号（第二个域）就用来形成物理地址；如果不匹配，那么就对链表中的下一个节点进行比较，以寻找一个匹配的页号。该方案如图 8.16 所示。

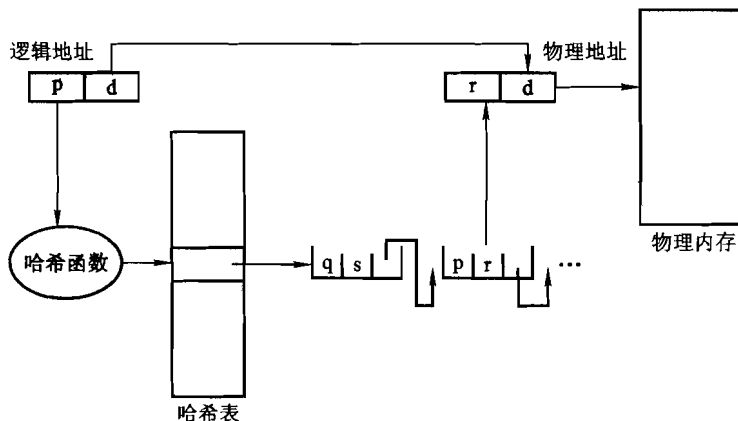


图 8.16 哈希页表

人们提出了这种方法的一个变种，它比较适合 64 位的地址空间。**群集页表**（clustered page table）类似于哈希页表，不过这种哈希页表的每一条目不只包括一页信息，而是包括多页（例如 16）。因此，一个页表条目可以存储多个物理页帧的映射。群集页表对于**稀疏**地址空间特别有用，稀疏地址空间中的地址引用不连续，且分散在整个地址空间。

8.5.3 反向页表

通常，每个进程都有一个相关页表。该进程所使用的每个页都在页表中有一项（或者每个虚拟地址都有一项，不管后者是否有效）。这种页的表示方式比较自然，这是因为进程是通过页的虚拟地址来引用页的。操作系统必须将这种引用转换成物理内存地址。由于页表是按虚拟地址排序的，操作系统能够计算出所对应条目在页表中的位置，并可以直接使用该值。这种方法的缺点之一是每个页表可能有很多项。这些表可能消耗大量物理内存，却仅用来跟踪物理内存是如何使用的。

为了解决这个问题，可以使用**反向页表**（inverted page table）。反向页表对于每个真正的内存页或帧才有一个条目。每个条目包含保存在真正内存位置的页的虚拟地址以及拥有该页的进程的信息。因此，整个系统只有一个页表，对每个物理内存的页只有一条相应的

条目。图 8.17 说明了反向页表的操作，将它与图 8.7 的标准页表操作相比较。因为系统只有一个页表，而有多个地址空间映射物理内存，所以反向页表的条目中通常需要一个地址空间标识符（见 8.4.2 小节），以确保一个特定进程的一个逻辑页可以映射到相应的物理帧。采用反向页表的系统包括 64 位的 UltraSPARC 和 PowerPC。

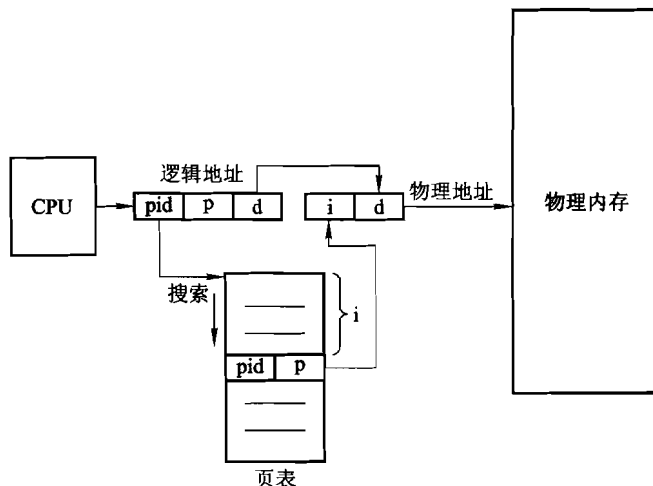


图 8.17 反向页表

为了说明这种方法，这里描述一种简化的反向页表实现，IBM RT 就使用这种方法。系统的每个虚拟地址有一个三元组

$\langle \text{process-id}, \text{page-number}, \text{offset} \rangle$

每个反向页表的条目为一对 $\langle \text{process-id}, \text{page-number} \rangle$ ，其中 **process-id** 用来作为地址空间的标识符。当需要内存引用时，由 $\langle \text{process-id}, \text{page-number} \rangle$ 组成的虚拟地址部分送交内存子系统。通过查找反向页表来寻找匹配。如果匹配找到，例如条目 *i*，那么就产生了物理地址 $\langle i, \text{offset} \rangle$ 。如果没有匹配，那么就是试图进行非法地址访问。

虽然这种方案减少了存储每个页表所需要的内存空间，但是当引用页时，它增加了查找页表所需要的时间。由于反向页表按物理地址排序，而查找是根据虚拟地址，因此可能需要查找整个表来寻求匹配。这种查找会花费很长时间。为了解决这一问题，可以使用 8.5.2 小节介绍的哈希页表来将查找限制在一个或少数几个页表条目。当然，每次访问哈希页表也为整个过程增加了一次内存引用，因此一次虚拟地址引用至少需要两个内存读：一个查找哈希页表条目，另一个查找页表。为了改善性能，可以在访问哈希页表时先查找 TLB。

采用反向页表的系统在实现共享内存时存在困难。共享内存通常作为被映射到一个物理地址的多虚拟地址（其中每一个进程共享内存）来实现。这种标准的方法不能用到反向

页表，因为此时每个物理页只有一个虚拟页条目，一个物理页不可能有两个（或更多）的共享虚拟地址。解决该问题的一个简单方法是允许页表仅包含一个虚拟地址到共享物理地址的映射，这意味着对未被映射的虚拟地址的引用将导致页错误。

8.6 分 段

采用分页内存管理有一个不可避免的问题：用户视角的内存和实际物理内存的分离。前面已经看到，用户视角的内存与实际物理内存不一样。用户视角的内存需要映射到实际物理内存，该映射允许区分逻辑内存和物理内存。

8.6.1 基本方法

用户是否会将内存看做是一个线性字节数组，有的包含指令而其他的包含数据？绝大多数人会说不。用户通常愿意将内存看做是一组不同长度的段的集合，这些段之间并没有一定的顺序（见图 8.18）。

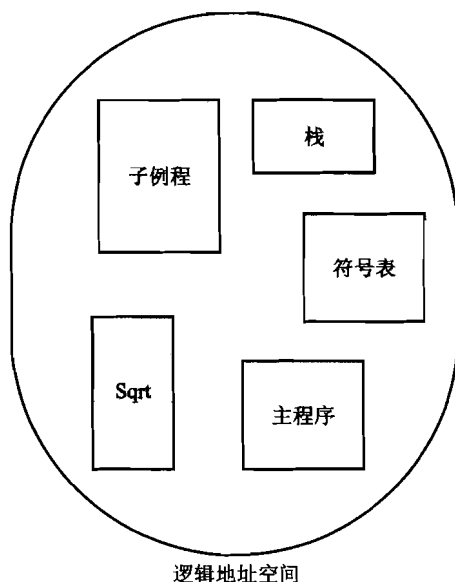


图 8.18 用户眼中的程序

想一下你在写程序时是如何考虑程序的。人们会认为程序是由主程序加上一组方法、过程、函数所构成的，还有各种数据结构：对象、数组、堆栈、变量等。每个模块或其他数据元素都可通过名称引用。人们会说“堆栈”、“数学库”、“主程序”，而并不关心这些元

素所在内存的位置，不关心栈是放在函数 `Sqrt()` 之前还是之后。这些段的长度是不同的，其长度是由这些段在程序中的目的所定的。段内的元素是通过它们距段首的偏移来指定：程序的第一条语句、栈里的第 7 个栈帧、函数 `Sqrt()` 的第 5 条指令等。

分段(segmentation)就是支持这种用户视角的内存管理方案。逻辑地址空间是由一组段组成的。每个段都有名称和长度。地址指定了段名称和段内偏移。因此用户通过两个量来指定地址：段名称和偏移（请将这一方案与分页相比较。在分页中，用户只指定一个地址，该地址通过硬件分为页码和偏移，对于这些，程序员是看不见的）。

为实现简单起见，段是编号的，是通过段号而不是段名来引用的。因此，逻辑地址由有序对组成：

`<segment-number, offset>`

通常，在编译用户程序时，编译器会自动根据输入程序来构造段。

一个 C 编译器可能会创建如下段：

- ① 代码。
- ② 全局变量。
- ③ 堆（内存从堆上分配）。
- ④ 每个线程采用的栈。
- ⑤ 标准的 C 库函数。

在编译时链接的库可能被分配不同的段。加载程序会装入所有这些段，并为它们分配段号。

8.6.2 硬件

虽然用户现在能够通过二维地址来引用程序中的对象，但是实际物理内存仍然是一维序列的字节。因此，必须定义一个实现方式，以便将二维的用户定义地址映射为一维物理地址。这个地址是通过**段表(segment table)**来实现的。段表的每个条目都有**段基地址**和**段界限**。段基地址包含该段在内存中的开始物理地址，而段界限指定该段的长度。

段表的使用如图 8.19 所示。一个逻辑地址由两部分组成：段号 s 和段内的偏移 d 。段号用做段表的索引，逻辑地址的偏移 d 应位于 0 和段界限之间。如果不是这样，会陷入到操作系统中（逻辑地址试图访问段的外面）。如果偏移 d 合法，那么就与段基地址相加而得到所需字节在物理内存的地址。因此段表是一组基地址和界限寄存器对。

如图 8.20 所示，有 5 个段，编号为 0~4。各段按图中所示存储。每个段都在段表中有一个条目，它包括段在物理内存内的开始地址（或基地址）和该段的长度（或界限）。例如，段 2 为 400 B 长，开始于位置 4300。因此，对段 2 第 53 字节的引用映射成位置 $4300+53=4353$ 。对段 3 第 852 字节的引用映射成位置 $3200+852=4052$ 。段 0 第 1222 字节的引用会陷入操作系统，这是由于该段仅为 1 000 B 长。

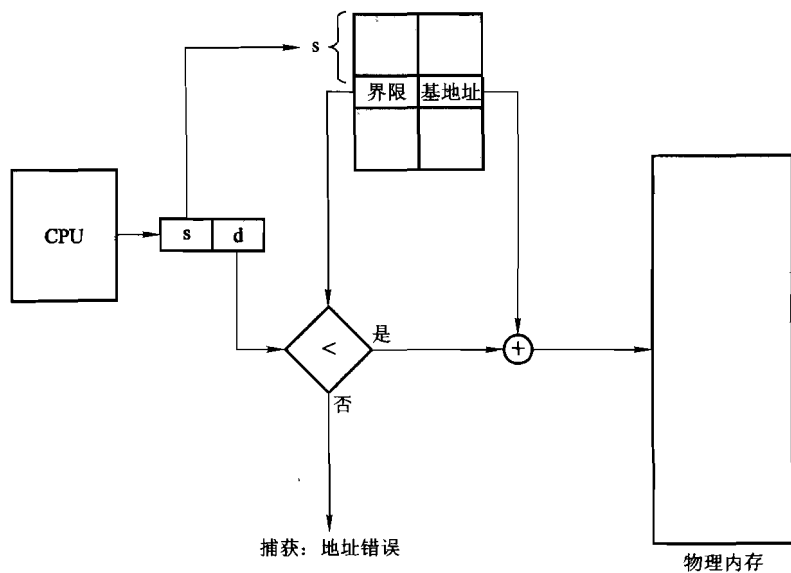


图 8.19 分段硬件

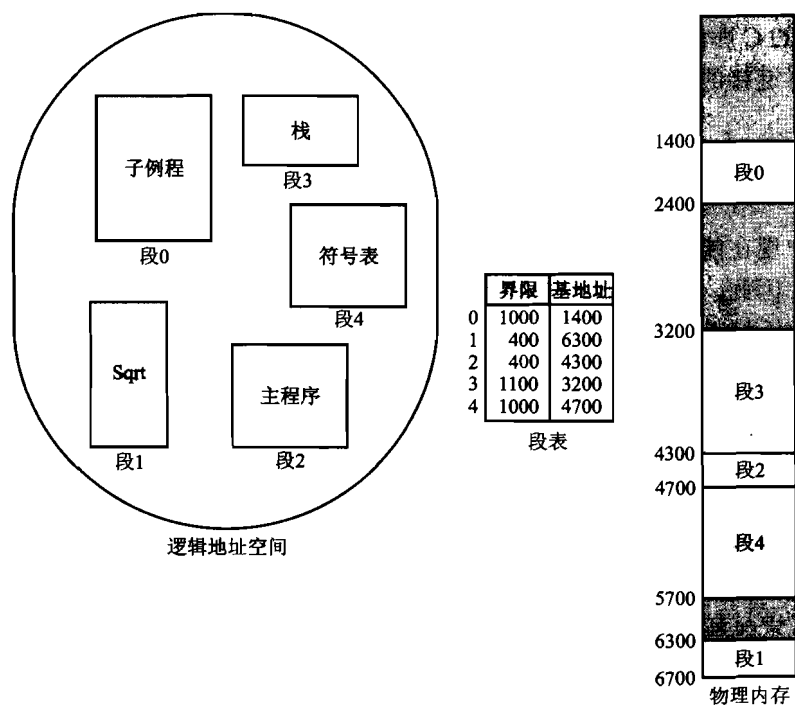


图 8.20 分段的例子

8.7 实例: Intel Pentium

分页或分段都有其优缺点。事实上,有些体系结构两种都提供。本节将讨论支持单纯的分段或分页加分段的 Intel Pentium 体系结构。在此并不会给出对 Pentium 内存管理结构的完整介绍,而是介绍它所基于的主要思想。最后以对 Pentium 系统上 Linux 地址转换概述作为讨论的结束。

在 Pentium 系统中, CPU 产生逻辑地址,它被赋给分段单元。分段单元为每个逻辑地址生成线性地址,然后线性地址交给分页单元,它接下来生成内存中的物理地址。因此,分段单元和分页单元相当于内存管理单元(MMU)。这种方案如图 8.21 所示。

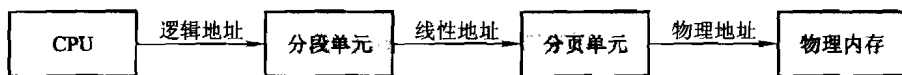


图 8.21 Pentium 中逻辑地址到物理地址的转换

8.7.1 Pentium 分段

Pentium 结构允许一个段的大小最多可达 4 GB,每个进程最多的段的数量为 16 K 个。进程的逻辑地址空间被分为两部分:第一个部分最多由 8 K 个段组成,这部分为私有;第二个部分最多由 8 K 个段组成,这部分为所有进程所共享。关于第一个部分的信息保存在本地描述符表(local descriptor table, LDT)中,而关于第二个部分的信息保存在全局描述符表(global descriptor table, GDT)中。LDT 和 GDT 的每个条目为 8 B,包括一个段的详细信息,如基位置和段界限等。

逻辑地址是一对(selector, offset),选择器(selector)是一个 16 位的数:



其中, s 表示段号, g 表示段是在 GDT 还是在 LDT 中, p 表示保护信息。偏移(offset)是一个 32 位的数,用来表示字节(或字)在段内的位置。

机器有 6 个段寄存器,允许一个进程可以同时访问 6 个段。它还有 6 个 8 B 的微程序寄存器,用来保存相应的来自于 LDT 或 GDT 的描述符。这一缓冲区允许 Pentium 不必在每次内存引用时都从内存中读取描述符。

Pentium 的线性地址为 32 位长,按如下方式来形成:段寄存器指向 LDT 或 GDT 中的适当条目,段的基地址和界限信息用来产生线性地址。首先,界限用来检查地址的合法性。如果地址无效,就产生内存出错,导致陷入操作系统。如果有效,偏移值就与基地址的值

相加，产生 32 位的线性地址，如图 8.22 所示。下一小节将讨论分页单元如何将线性地址转换成物理地址。

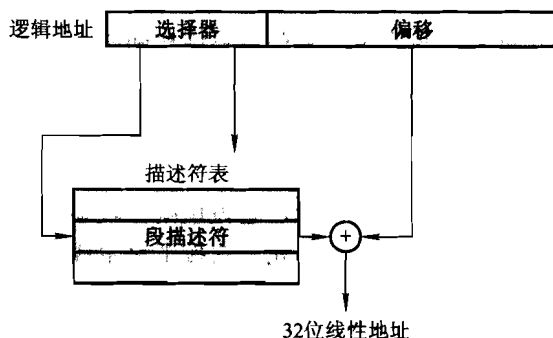
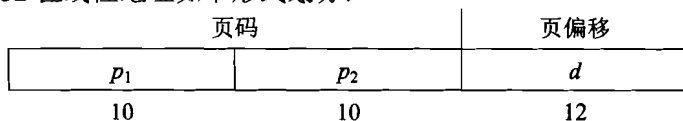


图 8.22 Intel Pentium 分段

8.7.2 Pentium 分页

Pentium 体系结构允许页的大小为 4 KB 或 4 MB。对于 4 KB 的页，Pentium 采用二级分页方案，其中 32 位线性地址如下形式划分：



这种体系结构的地址转换方案类似于图 8.15。Intel Pentium 地址转换的详细情况如图 8.23 所示。最高 10 位引用最外层页表的条目，它被称为页目录 (page directory) (CR3 寄存器指向当前进程的页目录)。页目录条目指向由线性地址中最内层的 10 位内容索引的内部页表。最后，最低的 0~11 位是页表项所指向的 4 KB 页面内的偏移。

页目录中的一个条目是 Page Size 标志，如果设置了它，表示页帧的大小为 4 MB，而不是标准的 4 KB。如果设置了该标志，则页目录直接指向 4 MB 页帧，而绕过了内层页表，且线性地址的最低 22 位指向 4 MB 页帧的偏移。

为了提高物理内存的使用效率，Intel Pentium 的页表可以交换到磁盘上。这时，页目录条目的无效位可用来表示相应的页表是在内存还是在磁盘上。如果在磁盘上，操作系统可以使用其他的 31 位来表示页表在磁盘上的位置，该页表可根据需要调入内存。

8.7.3 Pentium 系统上的 Linux

作为一个例子，考虑运行在 Intel Pentium 体系结构上的 Linux 系统。由于 Linux 被设计为在一系列处理器上运行（其中许多可能仅提供对段的有限支持）Linux 并不依赖段，

并最低程度地使用它。在 Pentium 中, Linux 仅使用 6 个段:

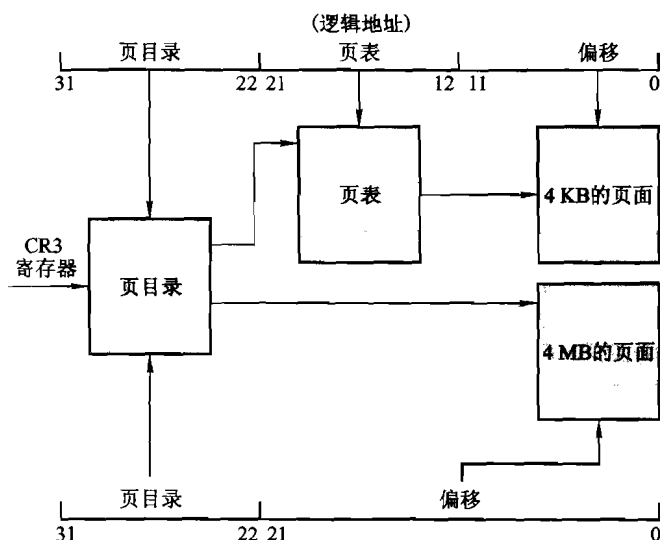


图 8.23 Pentium 体系结构中的分页

- 内核代码段。
- 内核数据段。
- 用户代码段。
- 用户数据段。
- 任务状态段 (TSS)。
- 默认的 LDT 段。

用户代码段和用户数据段为所有以用户模式运行的进程所共享。由于所有进程使用相同的逻辑地址空间且所有段描述符保存在全局描述符表 (GDT) 内, 因此这是可能的。进一步讲, 每个进程都有它自己的任务状态段 (TSS), 该段的描述符被保存在 GDT 中。该 TSS 被用来保存在上下文切换中每个进程的硬件上下文。通常, 默认的 LDT 段被所有进程所共享, 且通常并不被使用。不过, 如果一个进程需要它自己的 LDT, 则它可以生成一个 LDT, 并用它代替默认的 LDT。

正如所指出的, 每个段选择器包括一个 2 位保护域, 因此 Pentium 允许四级保护。在此四级保护中, Linux 仅识别用户模式与内核模式。

尽管 Pentium 采用二级分页模式, Linux 被设计为能运行多种硬件平台, 其中许多平台是 64 位的, 而二级分页对此并不合适。因此, Linux 采用适合 32 位和 64 位体系结构的三级分页方案。

Linux 中的线性地址分成如下 4 个部分：



图 8.24 中明确了 Linux 中的三级分页模式。

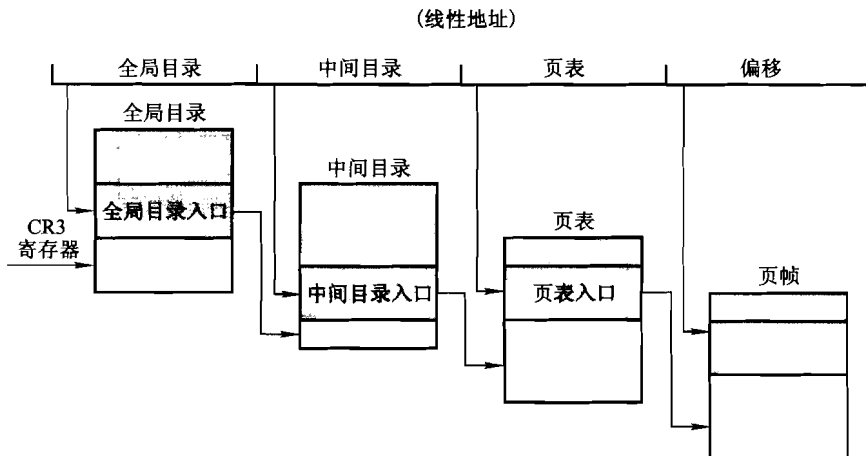


图 8.24 Linux 中的三级分页模式

根据体系结构的不同，线性地址的每一部分的位的多少也有变化。但是，正如本节前面所讲的，Pentium 体系结构仅采用二级分页模式。那 Linux 是如何在 Pentium 上使用三级模式的呢？此时，中间目录的大小为零位，实际上绕开了中间目录。

Linux 中的每个任务都有它自己的页表，如图 8.23 所示，CR3 寄存器指向了当前执行的任务的全局目录。在上下文切换中，CR3 寄存器的值被保存和恢复是在与上下文切换相关任务的 TSS 段中进行的。

8.8 小 结

用于多道程序设计的操作系统的内存管理算法，包括从简单的单用户系统方法到分页分段方法。一个特定系统所采用方法的最大决定因素是硬件所提供的支持。CPU 所产生的每个内存地址都必须先进行合法性检查，才可映射成为物理地址。检查是不能通过软件来（有效地）实现，因此实现受到可用硬件的限制。

不同的内存管理算法（连续分配、分页、分段，以及分页与分段结合）在许多方面都呈现出不同的特点。在比较不同内存管理策略时，需要考虑如下几点：

- **硬件支持：**对单分区和多分区方案，只需要一个基地址寄存器或一对基地址和界限

寄存器就够了；而对于分页和分段，需要映射表以定义地址映射。

- **性能：**随着内存管理算法变得越来越复杂，逻辑地址到物理地址的映射所需要的时间也有所增加。对于简单系统，只需要对逻辑地址进行比较和加减操作（这些较为简单）。如果表能通过快速寄存器来加以实现，那么分页和分段操作也会很快。不过，如果表在内存中，那么用户内存访问速度就大大地降低。TLB 可以用来改善性能。

- **碎片：**如果多道程序的级别更高，那么多道程序系统通常会更有效地执行。对于给定的一组进程，可以通过将更多进程装入内存以增加多道程序的程度。为了完成这一任务，必须降低内存浪费或碎片。采用固定大小分配单元（如单个分区和分页）的系统会有内部碎片问题。采用可变大小分配单元（如多个分区和分段）的系统会有外部碎片问题。

- **重定位：**外部碎片问题的解决方案之一是紧缩。紧缩涉及在内存中移动程序而不影响到程序。这种方案要求在执行时逻辑地址能动态地进行重定位。如果地址只能在装入时进行重定位，那么就不能采用紧缩。

- **交换：**任何算法都可加上交换操作。进程可以定时地（由操作系统来定，通常由 CPU 调度策略来决定）从内存交换到外存，之后再交换到内存。这种方法能允许更多进程运行，但它们不能同时装入内存。

- **共享：**另一个增加多道程序程度的方法是让不同用户共享代码和数据。共享通常要求分页或分段，以便共享较小的信息区域（如页或段）。共享在有限内存情况下能运行许多进程，但是共享的程序和数据需要仔细设计。

- **保护：**如果提供了分页或分段，那么用户程序的不同区域可以声明为只可执行、只读或可读写。这种限制对于共享代码和数据是必要的，对于常见程序设计错误能提供简单的运行时的检查。

习 题

8.1 试说明内部碎片与外部碎片的区别。

8.2 考虑下面生成二进制的过程。编译器被用来生成单个单元的目标代码，链接器被用来将多个目标单元合并为一个程序二进制。链接器如何改变指令和数据到内存地址的绑定？需要将什么信息从编译器传递给链接器，以协助完成链接器的内存绑定任务？

8.3 如果有内存块 100 KB、500 KB、200 KB、300 KB 和 600 KB（按顺序），首次适应算法、最佳适应算法、最差适应算法各自将怎样放置大小分别为 212 KB、417 KB、112 KB 和 426 KB（按顺序）的进程？哪一种算法的内存利用率最高？

8.4 绝大多数系统允许程序在执行时分配更多的内存给它自己的地址空间。程序的堆段中的数据分配就是这样一个内存分配实例。在下面的方法中，支持动态内存分配需要什么？

- 连续内存分配
- 纯分段

c. 纯分页

8.5 对下列问题，试比较连续内存分配方案、纯分段方案和纯分页方案中的内存组织方法：

a. 外部碎片

b. 内部碎片

c. 共享跨进程代码的能力

8.6 在一个分页系统中，进程不能访问不属于它的内存，为什么？操作系统怎样才能允许访问其他的内存？应不应该，为什么？

8.7 试比较分页与分段在将虚拟地址转换到物理地址过程中地址转换结构所需要的内存量。

8.8 许多系统中，程序二进制常见的结构如下：代码从一个小的固定虚拟地址（如 0）开始保存。代码段后是用来保存程序变量的数据段，当程序开始执行时，栈在虚拟地址空间的另一端被分配，并被允许向更低的虚拟地址方向递增。在下列方案中，采用上述结构有何意义？

a. 连续内存分配

b. 纯分段

c. 纯分页

8.9 假设一个将页表存放在内存的分页系统：

a. 如果一次内存访问需 200 ns，访问一页内存要用多长时间？

b. 如果加入 TLB，并且 75% 的页表引用发生在 TLB，内存有效访问时间是多少？（假设在 TLB 中查找页表项占用零时间，如果页表项在其中。）

8.10 为什么时常将分页与分段在同一个方案中结合使用？

8.11 试说明为什么使用分段比使用纯分页更容易共享一个可重入模块。

8.12 假设有下面的段表：

<u>段</u>	<u>基地址</u>	<u>长度</u>
0	219	600
1	2300	14
2	90	100
3	1327	580
4	1952	96

下面逻辑地址的物理地址是多少？

a. 0430

b. 110

c. 2500

d. 3400

e. 4122

8.13 页表分页的目的是什么？

8.14 考虑 VAX 体系结构所采用的多级分页方案，当用户程序执行一个内存装载操作时，完成了多少内存操作？

8.15 比较一下在处理大型地址空间时的分段分页方案与哈希页表方案。两者分别适合什么环境？

8.16 考虑如图 8.22 所示的 Intel 地址转换方案：

a. 描述 Intel Pentium 将逻辑地址转换成物理地址的所有步骤。

- b. 使用这样复杂的地址转换方案对操作系统有什么好处？
- c. 这样的地址转换系统有没有什么缺点？如果有，有哪些？如果没有，为什么不是每个制造商都使用这种方案？

文献注记

Knuth[1973]论述了动态内存分配（参见 2.5 节），他还从模拟结果中发现首次适应分配算法比最佳适应算法好。Knuth[1973]讨论了 50%规则。

分页的概念要归功于 Atlas 系统的设计者们，Kilburn 等[1961]和 Howarth 等[1961]描述了这些。Dennis[1965]论述了分段的概念。GE 645 最先支持分页分段，在它上面最初实现 MULTICS（Organick[1972]、Daley 和 Dennis[1967]）。

Chang 和 Mergen[1988]在关于 IBM RT 的存储管理的论文中讨论了反向页表。

Jacob 和 Mudge[1997]讨论了有关地址转换的软件方法。

Hennessy 和 Patterson[2002]讨论了 TLB、Cache、MMU 的硬件特性。Talluri 等[1995]讨论了 64 位地址空间的页表。Wahbe 等[1993a]、Chase 等[1994]、Bershad 等[1995a]和 Thorn[1997]提出并研究了另一种强制内存保护的可选方法。Dougan 等[1999]、Jacob 和 Mudge[2001]讨论了一种管理 TLB 的技巧。Fang 等[2001]评估了对大页面的支持。

Tanenbaum[2001]讨论了 Intel 80386 的分页。Jacob 和 Mudge[1998a]描述了多种体系结构的内存管理，像 Pentium II、PowerPC 和 UltraSPARC。Bovet 和 Cesati[2002]提出了 Linux 系统的分段问题。